nornRTOS

User Guide

A real time operating system for small microcontrollers:
the nornCore kernel, nornNET and nornWIFI

Kernel `RTOS_VERSION` 2.0.0-dev, measurements of 2026-11-09

Tenforward AB

November 11, 2026

Licence. nornRTOS and this guide are licensed under the Business Source License 1.1 with Tenforward AB's Additional Use Grant: production use in your own products is allowed, offering nornRTOS as an operating system, kernel or SDK of your own is not, and each release becomes Apache 2.0 four years after it is made. The full text is `LICENSE.md` in every repository. Third-party components keep their own licences (`THIRD_PARTY_LICENSES.md`).

Sources. `https://github.com/NornRTOS` holds the nornRTOS tree, the modules nornCore, nornNET and nornWIFI, and nornDocs, the repository this guide is built from. The numbers in this guide come from the measurement summary `step10-lifecycle-after` in nornCore's `doc/measurements/`; the graphs are regenerated from it by `tools/measurements_to_csv.py`.

Contents

1 Introduction **6**
- 1.1 What kind of kernel this is . . . 6
- 1.2 The modules and the boards . . . 7
- 1.3 Conventions . . . 7

2 Getting started **8**
- 2.1 Toolchains . . . 8
- 2.2 The source tree . . . 8
- 2.3 Building and running . . . 8
- 2.4 An application . . . 9
- 2.5 The smallest application . . . 9
- 2.6 Hooks . . . 11
- 2.7 The examples . . . 11

3 Concepts **12**
- 3.1 Tasks and priorities . . . 12
- 3.2 Scheduling . . . 12
- 3.3 Time . . . 13
- 3.4 Interrupts and calling contexts . . . 13
- 3.5 Blocking and hand-over . . . 14
- 3.6 Mutual exclusion . . . 14
  - 3.6.1 The problem . . . 14
  - 3.6.2 The immediate priority ceiling protocol . . . 14
  - 3.6.3 When the ceiling is wrong, or the owner blocks . . . 15
  - 3.6.4 The order at equal effective priority . . . 15
  - 3.6.5 Rules for using mutexes . . . 16
- 3.7 The task life cycle . . . 16
- 3.8 Stacks . . . 16

4 Configuration **18**
- 4.1 The options . . . 18
- 4.2 Three configurations to start from . . . 19
- 4.3 The configuration signature . . . 19
- 4.4 A priority map . . . 19

5 Using the primitives **21**
- 5.1 Task event flags . . . 21
- 5.2 Counting semaphores . . . 22
- 5.3 Ceiling mutexes . . . 22
- 5.4 Message queues . . . 23
- 5.5 Mailboxes . . . 23
- 5.6 Task notifications . . . 24

5.7 Software timers . . . 24
5.8 Memory pools . . . 24
5.9 Event groups . . . 25
5.10 Suspend and resume . . . 25
5.11 The heap . . . 25

6 Robustness and diagnostics **26**
6.1 Checks and assertions . . . 26
6.2 Panic and the record . . . 26
6.3 Stack checking . . . 27
6.4 The watchdog . . . 27
6.5 Deadlock detection . . . 27
6.6 Run time statistics and task information . . . 27
6.7 Trace . . . 27

7 API reference **28**
7.1 Constants and types . . . 29
7.2 Kernel life cycle . . . 29
7.3 Time and delays . . . 30
7.4 Task event flags . . . 31
7.5 Counting semaphores . . . 31
7.6 Ceiling mutexes . . . 32
7.7 Message queues . . . 32
7.8 Mailboxes . . . 33
7.9 Task notifications . . . 33
7.10 Software timers . . . 34
7.11 Memory pools . . . 34
7.12 Event groups . . . 35
7.13 Suspend and resume . . . 35
7.14 Heap . . . 36
7.15 Diagnostics . . . 36
7.16 Panic and reset cause . . . 37
7.17 Hooks . . . 37
7.18 Watchdog . . . 38

8 Networking: nornNET, nornWIFI and nornHTTP **39**
8.1 nornNET . . . 39
8.1.1 Attaching it to an application . . . 39
8.1.2 Sockets . . . 39
8.1.3 Links that come and go . . . 40
8.2 nornWIFI . . . 40
8.2.1 Using it . . . 40
8.3 nornHTTP . . . 41
8.3.1 Using it . . . 41
8.3.2 What it costs . . . 42

9 Performance **43**
9.1 Method . . . 43
9.2 The cost of a call . . . 43
9.3 The context switch . . . 44
9.4 Interrupt latency . . . 44
9.5 Memory . . . 47

9.6 Stacks 47
9.7 The cost of the configuration 47
9.8 History 47
9.9 The older timing test 47

10 Verification **49**
10.1 Host unit tests 49
10.2 Test applications on the targets 50
10.3 Continuous integration and the simulators 50

A Porting in brief **51**

B Towards dynamic tasks **52**

Bibliography **53**

Index of API functions **54**

Chapter 1

Introduction

nornRTOS is a real time operating system for small microcontrollers. It consists of a kernel, *nornCore*, a TCP/IP stack, *nornNET*, and a driver for the ESP32's WiFi radio that runs without ESP-IDF or FreeRTOS, *nornWIFI*. The kernel runs on the 8 bit AVR family (ATmega328P on the Arduino UNO, ATmega2560 on the Arduino Mega) and on the Xtensa LX6 of the ESP32; a host port runs the same kernel on Linux for testing. The whole system is written in C99 with a few dozen lines of assembler per port, and every piece of it is compiled from source together with the application and its configuration.

This guide is for the engineer who writes an application on nornRTOS. It explains the model of the kernel (chapter 3), how an application is built and configured (chapters 2 and 4), how the primitives are meant to be used (chapter 5) and what the kernel does when something goes wrong (chapter 6). Chapter 7 is the reference of the public API, chapter 8 covers the network modules, and chapters 9 and 10 present the measurements and the test suite the figures in this guide rest on. The form of the guide follows the RTuinOS user guide [1], the operating system nornRTOS grew out of and replaced.

1.1 What kind of kernel this is

nornCore is a *static*, *fixed priority*, *preemptive* kernel with *one task per priority*:

- **Static.** Every task is created in `main()` before the scheduler starts and lives for ever. The kernel never allocates memory: the application owns every stack and every synchronisation object. Nothing in the kernel grows at run time, so the memory an application needs is known when it is linked.
- **Fixed priority, one task per priority.** A task's identifier *is* its priority, 1 to `RTOS_CFG_NUM_TASKS`; 0 is the idle task. Two tasks never share a priority, so there is no round robin, no time slice and no question about the order in which equal tasks run. The theory behind fixed priority scheduling and its schedulability tests is Liu and Layland's [2] and its descendants, and nornCore gives the application exactly the model that theory assumes.
- **Preemptive, $O(1)$.** The scheduler is one bit mask of ready tasks and one highest-bit lookup: between four and twenty cycles depending on the CPU. A task that becomes ready with a higher priority than the running one runs at once; a task that blocks hands the CPU to the highest ready one. Every blocking call has a timeout.
- **Priority ceiling mutexes.** Mutual exclusion uses the immediate priority ceiling protocol [3, 4]: a task that locks a mutex runs at the mutex's ceiling while it holds it, so with a correct ceiling nobody ever blocks on a mutex, priority inversion is bounded by one critical section, and deadlock is impossible.
- **Measured.** The cost of every kernel call, the context switch, the interrupt latency and the stack each task needs are measured in CPU cycles on the real boards and in a cycle

exact simulator, before and after every change to the kernel (chapter 9). A kernel of a few kilobytes has no excuse for unknown numbers.

What is deliberately absent is as telling: time slicing, dynamic creation and deletion of tasks at run time (the life cycle in section 3.7 and appendix B are the preparation), priority inheritance (the ceiling protocol is simpler and has better bounds) and symmetric multiprocessing (the two ESP32 cores run two independent kernels joined by a message link instead).

1.2 The modules and the boards

Module	What it is	Boards
nornCore	the kernel, the ports, a small HAL (UART, GPIO, delay, trace)	UNO, Mega 2560, ESP32, host
nornNET	Ethernet, ARP, IPv4, ICMP, UDP, TCP, DHCP client, DNS resolver	ESP32, Mega 2560 (SLIP), hos
nornWIFI	the ESP32 radio as a station, WPA2 personal, a nornNET device	ESP32
nornHTTP	an HTTP/1.1 client over any byte stream, no heap, streamed bodies	any, with a nornNET adapter

Table 1.1: The modules of nornRTOS.

Board	CPU	Clock	RAM	Flash	Simulator
Arduino UNO	ATmega328P, 8 bit AVR	16 MHz	2 KB	32 KB	simavr (cycle exact)
Arduino Mega 2560	ATmega2560, 8 bit AVR	16 MHz	8 KB	256 KB	simavr (cycle exact)
ESP32 devkit	ESP32-D0WDQ6, 2 Xtensa LX6	240 MHz	520 KB	4 MB	Espressif QEMU (not cycle

Table 1.2: The boards. The ESP32 runs bare metal: the ROM loads the image, no ESP-IDF.

The UNO has room for the kernel and a small application; nornNET runs on the Mega over a serial line and on the ESP32 over Ethernet (QEMU) or WiFi (nornWIFI). Everything in this guide that gives a number says which board it was measured on.

1.3 Conventions

Checked means a mandatory check, `RTOS_CHECK`, compiled into every configuration. A violation stops the system with `RTOS_PANIC_ARGUMENT` and a record naming the values. Task ids, NULL storage, stacks below the minimum, timeouts above the limit and foreign blocks freed are checked.

Asserted means `RTOS_ASSERT`, compiled only with `RTOS_CFG_ASSERT`. Without it a broken asserted contract is undefined behaviour. Recursive locks, unlock order and empty event masks are asserted.

Task context is a task function or anything it calls. **ISR body** is the code inside `RTOS_ISR`, a software timer callback running in the tick, or the trace hook. The **idle task** is the context of `main()` after `rtos_start()` and of the idle hook. Which functions each context may call is the subject of section 3.4.

Ticks are the kernel's unit of time, `RTOS_CFG_TICK_HZ` per second (1 kHz in every configuration shipped). `RTOS_MS(ms)` converts milliseconds.

Cycles are CPU clock cycles: 62.5 ns on the 16 MHz AVR, 4.17 ns on the 240 MHz ESP32. Most figures are in cycles because that is what the benchmark measures and what does not change with the clock.

Function names are written as in the header: `rtos_semGive`. A function with the suffix `FromIsr` is the variant for an ISR body. Names from the configuration start with `RTOS_CFG_`.

Chapter 2

Getting started

2.1 Toolchains

AVR `avr-gcc`, `avr-libc`, `avrdude` for uploading and `simavr` for running an image on the host. simavr is cycle exact; every example and every test case runs in it with the same counters as on the board.

ESP32 Espressif's `xtensa-esp-elf` GCC (the variable `XTENSA_ROOT` points at it), `esptool` in a Python environment (`ESP_PYTHON`) and, optionally, Espressif's fork of QEMU (`QEMU_ESP`) which runs every example but the WiFi ones. Nothing of ESP-IDF is needed: the kernel brings its own start-up, linker scripts and image format.

Host a C compiler and `make`. The host port runs the kernel on `ucontext` and simulates interrupts, which is how the unit tests run.

2.2 The source tree

```
nornRTOS/
  modules/        nornCore, nornNET, nornWIFI    (git submodules)
  boards/         what each board is and has: esp32, avr (mega2560, uno)
  examples/       hello, semaphore, basics, timing, bench, net, wifi
  tools/          submodules.sh, monitor.py, qemu_run.py, bin2c.py
  Makefile        builds any application from the module sources
nornCore/
  kernel/         rtos.h (the API), rtos_types.h, rtos_kernel.c, the primitives
  port/<arch>/    avr, xtensa, host: port.h, context switch, tick, start-up
  hal/            uart.h, gpio.h, delay.h, trace.h, testresult.h, board.h per arch
  config/         rtos_config.template.h, every option documented
  applications/   tc01..tc12, the test cases that run on the boards
  test/           test_*.c, the host unit tests
  tools/          test_all.sh, measure.sh, host_cov.sh, analyze.sh
  doc/            development material: plans, audits, measurements
```

Clone the tree with its submodules (`git clone -recursive`, or `tools/submodules.sh` afterwards). The public documentation, including this guide, is the separate repository nornDocs.

2.3 Building and running

From the nornRTOS directory:

```
make APP=hello BOARD=esp32                          image in modules/nornCore/build/
    hello-esp32
```

```
make APP=hello BOARD=esp32 sim                    run it in Espressif's QEMU
make APP=basics BOARD=mega2560 sim                run it in simavr
make APP=hello BOARD=uno upload PORT=/dev/ttyACM0
make APP=wifi BOARD=esp32 upload PORT=/dev/ttyUSB0
make APP=wifi BOARD=esp32 monitor                 serial console
make APP=myapp APPDIR=/path/to/myapp              an application kept elsewhere
make test                                         nornCore's whole suite
```

`BOARD` is `uno`, `mega2560` or `esp32`. The build is nornCore's Makefile with the tree passed in; `PORT`, `EXTRA_CFLAGS`, `SIM_SECONDS` and `CPU_MHZ` pass through. On the AVR boards the Arduino bootloader takes the image over the serial port; on the ESP32 `esptool` writes it to flash offset `0x1000`, from where the ROM loads it directly.

2.4 An application

An application is a directory with its sources and two files:

`app.mk` the default `BOARD`, the modules it uses (`MODULES`) and, on the ESP32, which objects run from flash instead of IRAM (`FLASH_OBJS`).

`rtos_config.h` the kernel configuration, started from `config/rtos_config.template.h` (chapter 4). Applications that use nornNET or nornWIFI add `nn_config.h` and `nw_config.h`.

The kernel and the modules are compiled from source with exactly that configuration, so an application never links against a kernel built for another one. Should it happen anyway (a stale build directory), `rtos_init` notices: every layout changing option is folded into `RTOS_CONFIG_SIGNATURE`, and a mismatch is `RTOS_PANIC_CONFIG` before any memory is touched.

2.5 The smallest application

`examples/hello` is one task that prints a line once a second and toggles the LED, with the idle hook counting idle loops and a panic hook that says what happened. It is also the skeleton of every nornRTOS application: hooks, `rtos_init`, `rtos_createTask` for each task, `rtos_start`.

Listing 2.1: examples/hello/hello.c

```
/**
 * @file hello.c
 * The smallest nornRTOS application: one task that prints a line once a second
    and
 * toggles the board's LED, the idle hook counting idle loops. Build it from the
    nornRTOS
 * directory: make APP=hello BOARD=esp32 sim.
 */

#include "rtos.h"
#include "uart.h"
#include "board.h"

static uint8_t _stack[RTOS_STACK_DEFAULT];
static const gpio_pin_t _led = BOARD_PIN_LED;
static volatile uint32_t _idleLoops;

void rtos_idleHook(void)
```

```
{
    ++ _idleLoops;
}

static void task(void *arg)
{
    uint32_t n = 0;
    (void)arg;
    gpio_output(&_led);
    for(;;)
    {
        (void)rtos_waitPeriodic(RTOS_MS(1000));
        gpio_toggle(&_led);
        uart_puts("hello from nornRTOS, second ");
        uart_putU32(++n);
        uart_puts(", idle loops ");
        uart_putU32(_idleLoops);
        uart_puts("\r\n");
    }
}

/** The panic hook as every application should have one: say what happened before
    the
    halt (the kernel's default is silent). */
void rtos_panicHook(const rtos_panicInfo_t *pInfo)
{
    uart_puts("\r\nPANIC reason "); uart_putU32(pInfo->reason);
    uart_puts(" task "); uart_putU32(pInfo->task);
    uart_puts(" arg1 "); uart_putU32(pInfo->arg1);
    uart_puts(" arg2 "); uart_putU32(pInfo->arg2);
    uart_puts(" (reason 6 = built against another configuration than the kernel)\r
    \n");
    uart_flush();
}

int main(void)
{
    uart_init(115200);
    uart_puts("\r\nnornRTOS hello\r\n");
    rtos_init();
    rtos_createTask(1, task, NULL, _stack, sizeof(_stack));
    rtos_start();
    return 0;
}
```

Four things in it deserve a word.

1. The stack is a static array of `RTOS_STACK_DEFAULT` bytes (192 on the AVR, 1024 on the ESP32). The kernel fills it with a pattern at creation, and `rtos_getStackFree` or the stack monitor read the high water mark later (section 3.8).
2. `rtos_waitPeriodic(RTOS_MS(1000))` is a drift free period: each boundary is the previous one plus the period, however long the task's work took. A late boundary is counted as an overrun, not lost.
3. The hooks are ordinary functions with weak defaults in the kernel: `rtos_idleHook`, `rtos_panicHook`, `rtos_stackWarnHook`, `rtos_assertFailed`, `rtos_traceHook`. Define the ones you want.
4. `rtos_start` never returns. The calling context becomes the idle task: it runs the idle

hook, the watchdog and the stack monitor when configured, and sleeps until the next interrupt.

2.6 Hooks

Hook	When
`rtos_idleHook()`	every iteration of the idle loop; must not block
`rtos_panicHook(pInfo)`	from `rtos_panic`, interrupts disabled, before the halt or reset
`rtos_stackWarnHook(task, free)`	once per task whose untouched stack drops below `RTOS_PORT_STACK_WARN`
`rtos_assertFailed(file, line)`	a failed `RTOS_ASSERT`; must not return
`rtos_traceHook(event, a, b)`	every traced kernel event with `RTOS_CFG_TRACE`; interrupts disabled

Table 2.1: The hooks an application may define.

A panic hook that prints the record is the one hook every application should have: the kernel's default is silent. `examples/semaphore` shows all of them.

2.7 The examples

hello one periodic task, the LED and the UART.
semaphore two tasks, a semaphore, a software timer and a notification; every hook defined.
basics nornCore's tc01 as an application: a periodic task, events, a semaphore given from an interrupt, a ceiling mutex, the stack check.
timing tc04: switch time, interrupt latency and periodic jitter on the current board, in microseconds.
bench tc10: the kernel's cost in cycles (chapter 9).
net nornNET over QEMU's Ethernet device: DHCP, DNS, an HTTP GET.
wifi nornWIFI on the ESP32 board: join an access point, DHCP, TCP.

Each prints a `RESULT <name> PASS` or `FAIL` line at the end and halts, which is how `make sim` knows it is done and how the test runner collects the results.

Chapter 3

Concepts

This chapter is the model of the kernel: what a task is, when it runs, how time is counted, what an interrupt may do, and how mutual exclusion works. Everything in the later chapters assumes it.

3.1 Tasks and priorities

A task is a C function that never returns, running on a stack the application provides, with a priority that is also its identifier. There are `RTOS_CFG_NUM_TASKS` application tasks, numbered 1 to `RTOS_CFG_NUM_TASKS` (at most 31), and the idle task 0, which is the context that called `rtos_start`. A higher number is a higher priority. Because the id is the priority, every kernel object that records waiting tasks does so as a bit mask (`rtos_taskmask_t`), and the highest waiter is the highest set bit.

The task control block (TCB) holds the saved context, the stack, the pending event flags, the wait state, the timeout and the statistics; it is 42 bytes on the AVR and 72 on the ESP32 in the default configuration (figure 9.4). The kernel adds nothing per task at run time.

3.2 Scheduling

The scheduler keeps one *ready mask* with a bit per task that is ready to run. The running task's bit stays set. The next task to run is the highest set bit: a lookup table on the AVR, one instruction on the Xtensa. A task that blocks clears its bit and the kernel switches to the highest remaining one; a task that is woken sets its bit, and if it is higher than the running task, the kernel switches to it at once (from task context) or at the end of the interrupt (from an ISR body).

The rule an application can rely on:

> At every instant the running task is the ready task with the greatest key (*effPrio*, *boosted*, *prio*), compared lexicographically. A ready task preempts the running one only if its key is strictly greater.

Without mutexes the effective priority equals the priority, no task is boosted, and the rule reduces to *the highest priority ready task runs.* The two other components matter only while a task holds a mutex whose ceiling is above its own priority (section 3.6); because priorities are unique, the order is total and no two tasks ever tie. There is no time slicing: a task runs until it blocks, is preempted by a higher task, or returns.

The idle task runs when no other task is ready. It executes the idle hook, the watchdog and the stack monitor when they are configured, and then, with `RTOS_CFG_IDLE_SLEEP`, puts the CPU to sleep until the next interrupt. With `RTOS_CFG_TICKLESS` on the ESP32 it also

stops the tick and sleeps until the next deadline. The idle task must never block; a blocking call from it is checked.

3.3 Time

The tick is a periodic interrupt at `RTOS_CFG_TICK_HZ`, 1 kHz in every shipped configuration. It increments the tick counter, wakes the tasks whose timeout has expired (they are kept in a list sorted by wake-up time, so the tick looks at the head only) and runs the software timers that are due. `rtos_getTick` returns the counter; it is 16 or 32 bits wide (`RTOS_CFG_TICK_BITS`) and wraps, so tick values are always compared as differences, never directly. The kernel does the same, and that is what limits every finite timeout to half the counter's range, `RTOS_MAX_TIMEOUT`: 32767 ticks with 16 bit ticks, $2^{31}-1$ with 32. A longer timeout is checked; `rtos_delay` is the one exception and splits a long delay into pieces.

Two things in the semantics of time are easy to get wrong:

- **A delay of n ticks lasts between n and $n+1$ tick periods**, because part of the current period is already over when the call is made. `rtos_delay(0)` waits for the next tick; it is not a yield (there is nothing to yield to: a task of the same priority does not exist).
- **A periodic task does not drift.** `rtos_waitPeriodic(period)` waits for the next boundary, and each boundary is the previous one plus the period, not the time of the call plus the period. If the boundary has passed, the call returns at once, counts an overrun and starts the next period from now. A task whose first iteration takes longer than a period therefore reports one overrun at start-up, harmlessly.

`rtos_getTimeUs` is a finer clock, read from a free running hardware counter (4 μs steps on the AVR, the cycle counter on the ESP32); it wraps after 71.6 minutes and is for measuring intervals.

3.4 Interrupts and calling contexts

An interrupt handler that talks to the kernel is written with the `RTOS_ISR` macro:

```
RTOS_ISR(BOARD_APP_TIMER_VECT)          /* a timer compare interrupt */
{
    rtos_semGiveFromIsr(&_sem);         /* wakes the consumer, no switch yet */
}
```

On the AVR the macro produces a naked handler that saves the caller clobbered registers, runs the body and ends in the kernel's *interrupt tail*: if the body made a higher task ready, the tail saves the rest of the context and switches, otherwise it returns to the interrupted task. On the ESP32 the macro defines the body as a function the port's exception dispatcher calls for that interrupt, and the tail is inlined into the dispatcher. In both cases the rule is the same: an ISR body never switches; *the switch happens once, at the end of the ISR*. That is what makes several wake-ups in one interrupt cost one switch, and it is why there are separate `FromIsr` functions: they do the same work as the task variants but defer the switch.

Interrupts do not nest on either port; a second interrupt waits and runs on the same spot. The longest interrupt-off window is therefore a context switch (about 27 μs on the Mega, 1.7 μs on the ESP32), and that is the worst case interrupt latency the benchmark in chapter 9 reports.

Critical sections. The kernel's own critical sections are short interrupt-off windows (`rtos_port_enterCritical` returns the previous state, `rtos_port_exitCritical` restores

Context	May call
Task	everything without a `FromIsr` suffix; a call may block or switch at once
ISR body, timer callback in the tick, trace hook	the `FromIsr` variants, `rtos_timerStart`, `rtos_timerStop`, `rtos_getTick`, `rtos_getTimeUs`, the readers (`rtos_queueCount`, `rtos_mboxIsFull`, `rtos_poolFreeCount`, `rtos_evgroupGet`, `rtos_timerIsActive`, `rtos_timerRemaining`) and `rtos_panic`; nothing blocks or switches
Idle task, idle hook	as a task, but nothing that blocks: waits only with `RTOS_NO_WAIT`; a blocking call is checked
Timer callback in the timer task	everything a task may call
Panic hook	nothing of the kernel; interrupts are disabled

Table 3.1: Calling contexts.

it, so they nest). An application may use the same pair for a few instructions of its own; it costs three cycles on the AVR and six on the ESP32. Everything longer belongs behind a mutex or inside a single task.

3.5 Blocking and hand-over

Every blocking call takes a timeout: `RTOS_NO_WAIT` polls, `RTOS_FOREVER` waits for good, anything else is ticks up to `RTOS_MAX_TIMEOUT`. A blocked task records what it waits on (`waitKind`, visible in `rtos_getTaskInfo`) and, if it has a timeout, sits in the timed list.

When a resource becomes available and a task waits for it, the kernel *hands it over* directly: a semaphore token goes to the highest waiter without touching the count, a queue item is copied straight into the receiver's buffer, a freed pool block goes to the waiting allocator, a mutex is passed to the highest waiter who becomes the owner at once. The resource never passes through the shared state, so a lower task cannot sneak in between the release and the wake-up, and the waiter's return value is decided at the moment of the release. The woken task runs before the releasing call returns if it is higher.

3.6 Mutual exclusion

3.6.1 The problem

A shared resource guarded by a plain lock produces *priority inversion*: a high task H blocks on a lock held by a low task L, and a medium task M that needs neither preempts L and keeps H waiting for as long as it likes. The classic remedies are priority inheritance, where L is raised to H's priority once H blocks, and the priority ceiling protocols, where the lock itself carries a priority [3].

3.6.2 The immediate priority ceiling protocol

nornCore implements the immediate variant, also known as the stack resource policy or highest locker protocol [4, 5]. Every mutex has a *ceiling*: the highest priority of any task that will ever lock it. Locking the mutex raises the caller's *effective priority* to the ceiling for as long as it holds the mutex; unlocking restores the previous one. The consequences, all of which the kernel relies on:

- **Nobody blocks on a mutex.** While L holds the mutex it runs at the ceiling, which is at least H's priority, so H cannot run and cannot find the mutex locked. The lock operation is a priority change and a few stores: 218 cycles for a lock and unlock pair on the Mega, 156 on the ESP32, with no scheduler involvement.
- **Inversion is bounded by one critical section.** H can be delayed by at most one lower task's critical section, and only one that was already running when H became ready.
- **Deadlock is impossible** with correct ceilings: a task that holds one mutex runs at a priority at which no other user of any mutex it will take next can hold it.
- **A medium task M with priority below the ceiling is held off** while L holds the mutex, even if M never uses it. That is the price of the protocol and the reason ceilings should be set exactly, not generously.

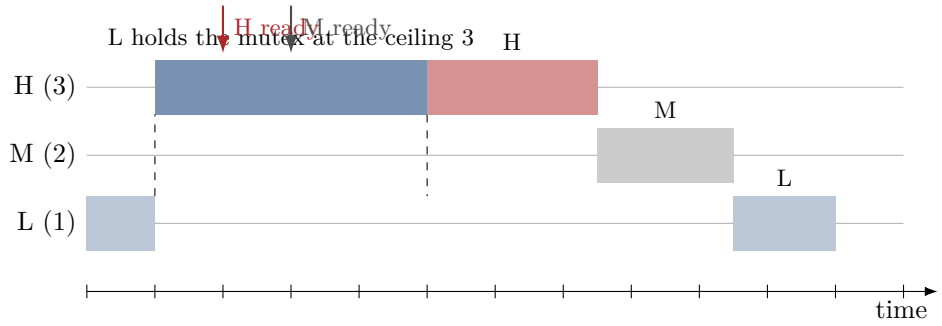


Figure 3.1: Immediate priority ceiling. L (priority 1) locks a mutex with ceiling 3 at $t = 1$ and runs at 3 until the unlock at $t = 5$. H (3) and M (2) become ready meanwhile and wait; H is delayed by one critical section of L and by nothing else, and M, which never uses the mutex, is held off as long as L holds it. Nobody ever blocks on the mutex itself.

3.6.3 When the ceiling is wrong, or the owner blocks

Two things break the ideal picture. A ceiling that is *too low* (a task above the ceiling locks the mutex) is a configuration error; the lock is asserted against it, and without assertions the task finds the mutex held. And an owner may *block while holding* the mutex (allowed, discouraged): it then leaves the CPU at the ceiling, and a task above its base priority but at or below the ceiling can run and try the lock.

For both cases the kernel has a fallback: the caller waits on the mutex, without a timeout, and the unlock hands the mutex to the highest waiter, who becomes the owner at the ceiling at once. A wait that would close a cycle of tasks waiting for each other's mutexes is detected and is `RTOS_PANIC_DEADLOCK`, naming both tasks. The detector follows chains of mutex waits only, which is why a task must not return while holding a mutex (section 3.7): an owner that is gone would make every later lock wait for ever without a cycle to detect.

3.6.4 The order at equal effective priority

While a task holds a mutex at the ceiling c, the task whose base priority is c may become ready. Both have effective priority c. The scheduling key from section 3.2 settles it: the *boosted* owner ranks above the task whose base priority is the ceiling, so the owner finishes its critical section first and the higher task does not preempt it. This is the choice that preserves the protocol's promise that nobody blocks: were the task at the ceiling to preempt, it would find the mutex locked. Two owners at one ceiling, which is possible only if one of them blocked

while holding, run in base priority order. The test `test/test_tie.c` pins every one of these cases down.

3.6.5 Rules for using mutexes

1. Set the ceiling to the highest priority of any task that locks the mutex, exactly.
2. Keep the critical section short; nothing below the ceiling runs meanwhile.
3. Do not block while holding a mutex. It is allowed, and the kernel copes, but it reintroduces waiting and it is the only way to get two tasks at one ceiling.
4. Lock in a fixed order and unlock in the reverse order (asserted). A mutex is not recursive (asserted).
5. Never return from a task function while holding a mutex (checked for every mutex whose ceiling is above the task's priority).
6. Do not lock from the idle task (asserted) or from an ISR body: an ISR body cannot wait, and a critical section is the right tool for a few instructions of shared state with an interrupt.

3.7 The task life cycle

A task slot is in one of five states, derived from the TCB rather than stored in it (no state byte, no store on the hot path), with suspension as a modifier on top of READY and BLOCKED:

State	Predicate	Entered by	Left by
UNUSED	no task function; the TCB is in its initial state	`rtos_init`, a future reap	`rtos_createTask`
READY	bit in the ready mask, not the running task	create, wake-up, preemption	pick, block, suspend
RUNNING	the current task (its bit stays in the ready mask)	pick	block, preemption, return
BLOCKED	bit not in the ready mask; `waitKind` says on what	a blocking call	wake-up, timeout
TERMINATED	`waitKind == RTOS_WAIT_TERMINATED`	the task function returning	a future reap

Table 3.2: The task states. `rtos_getTaskInfo` reports them as `RTOS_TASK_*`, with SUSPENDED for a suspended READY or BLOCKED task.

A task function *should* never return. If it does, the task is terminated: its slot and stack stay allocated, it is never scheduled again, and it is deaf to events, notifications, suspends and resumes aimed at it (they accumulate or are ignored harmlessly). Returning while holding a mutex is checked, for the reason given above. The kernel does not yet reuse a terminated slot at run time; the rules under which a future dynamic task API will do so are fixed (appendix B), and the test `test/test_lifecycle.c` already verifies them against the kernel.

3.8 Stacks

Each task runs on the stack given to `rtos_createTask`. What a task needs is its own frames plus what the kernel and the port leave under them when the task blocks or is interrupted. An interrupt lands on the stack of the running task, so every task must have room for the

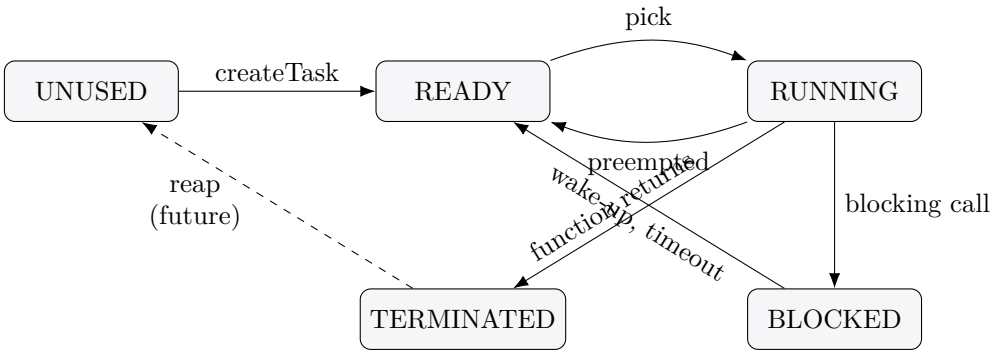


Figure 3.2: The life cycle of a task slot. Suspension (`rtos_suspend`) is a modifier on READY and BLOCKED, not a state: the task keeps its wait and its pending wake-ups and continues at `rtos_resume`.

interrupt frame and the kernel's work on top of its own deepest call chain. The port gives three sizes (table 3.3), and chapter 9 has the measurements they are derived from.

Name	ESP32	AVR	Meant for
`RTOS_STACK_MIN`	512 B	96 B	the technical minimum the kernel accepts (checked): very simple tasks whose use has been measured
`RTOS_STACK_DEFAULT`	1024 B	192 B	an ordinary task; start here
`RTOS_STACK_LARGE`	2048 B	384 B	a recommended start for tasks that print, nest calls or use floating point; not an upper bound

Table 3.3: The stack sizes the port names. Size a task from the default, run it, read the high water mark with `rtos_getStackFree`, and keep a margin above `RTOS_PORT_STACK_WARN` (96 B on the ESP32, 24 B on the AVR).

With `RTOS_CFG_STACK_CHECK` the stack is filled with a pattern at creation and the high water mark can be read; with `RTOS_CFG_STACK_WARN` the idle task watches one task per loop and calls `rtos_stackWarnHook` once for a task whose margin is gone; with `RTOS_CFG_STACK_CHECK 2` a canary at the bottom turns an actual overflow into `RTOS_PANIC_STACK_OVERFLOW`, checked before every switch and at every tick.

Chapter 4

Configuration

The kernel is configured at compile time by `rtos_config.h` in the application directory, started from `config/rtos_config.template.h`. Every option has a default (in `rtos_types.h`) so a configuration lists only what differs, with one exception: `RTOS_CFG_NUM_TASKS` must be given.

4.1 The options

Table 4.1: The configuration options, all prefixed `RTOS_CFG_`.

Option	Default	Meaning
`NUM_TASKS`	–	application tasks, 1 to 31; the idle task is not counted; each gets the priority equal to its id
`TICK_HZ`	1000	tick frequency; on the AVR it must divide $F_{CPU}/64$ exactly (1000 and 500 work at 16 MHz)
`TICK_BITS`	32	width of the tick counter, 16 or 32; 16 saves RAM on the UNO and limits timeouts to 32.7 s
`EVT_BITS`	16	width of the per task event flags, 8, 16 or 32; nornWIFI needs 32
`TIMER`	arch	AVR only: which 16 bit timer drives the tick (Mega: 1, 3, 4, 5; UNO: 1)
`STACK_CHECK`	1	1 fills stacks and supports `rtos_getStackFree`; 2 adds the canary; 0 drops the bookkeeping
`STACK_WARN`	1	the stack monitor in the idle task and `rtos_stackWarnHook`; needs `STACK_CHECK`
`IDLE_SLEEP`	1	the idle task sleeps between interrupts
`SW_TIMERS`	1	software timers; one list check per tick
`RUNTIME_STATS`	1	run time per task and CPU load, measured at every switch; about 45 cycles per switch on the AVR, 4 bytes per task
`ASSERT`	1	kernel assertions; about 20 cycles per switch on the AVR
`IDLE_HOOK`	–	a function called in every idle loop; the weak `rtos_idleHook` is the usual way
`PANIC_ACTION`	0	0 halts at a panic (development), 1 resets with the record kept for `rtos_getLastPanic`
`WATCHDOG`	0	hardware watchdog kicked by the idle task plus per task check-ins
`AVR_RESET_VIA_WDT`	0	AVR: reset through the watchdog (needs a bootloader that disables it) or by jumping to the reset vector
`TIMER_TASK_PRIO`	0	0 runs timer callbacks in the tick interrupt; a priority runs them in a kernel task of that priority (`TIMER_TASK_STACK` sizes it)
`TRACE`	0	call `rtos_traceHook` at switches, blocks, wake-ups, timers and panics

Option	Default	Meaning
EVGROUPS	1	event groups
NOTIFY	1	task notifications; 0 saves 2 bytes per task
SUSPEND	1	rtos_suspend and rtos_resume
HEAP	0	the first fit heap
TICKLESS	0	ESP32: the idle task stops the tick and sleeps until the next deadline

4.2 Three configurations to start from

The three sets in table 4.2 are measured starting points. *small* is the lean kernel for the UNO or for the smallest possible footprint; *default* is an ordinary application, and what nornWIFI needs on the ESP32; *large* turns on everything for development and diagnosis.

Option	small	default	large
NUM_TASKS esp32 / mega / uno	7 / 4 / 3	15 / 7 / 4	31 / 7 / –
TICK_BITS esp32 / avr	32 / 16	32 / 32 (uno 16)	32 / 32
EVT_BITS esp32 / avr	16 / 8	32 / 16 (uno 8)	32 / 16
STACK_CHECK	1	2 (canary)	2
STACK_WARN	0	1	1
RUNTIME_STATS	0	1	1
ASSERT	0	1	1
EVGROUPS, SUSPEND	0	1 (uno 0)	1
HEAP, WATCHDOG	0	esp32 1, avr 0	esp32 1, avr 0
TRACE	0	0	1
TICKLESS (esp32)	0	0	1
SW_TIMERS, NOTIFY	1 (uno: timers 0)	1	1
switch, cycles, esp32 / mega	347 / 315	440 / 472	472 / –

Table 4.2: Three configurations and their measured context switch (tc10, per switch including the signalling). Common to all: TICK_HZ 1000, IDLE_SLEEP 1, TIMER_TASK_PRIO 0, PANIC_ACTION 0.

Two constraints from the hardware: the original Mega 2560 bootloader does not survive a watchdog reset, so the watchdog stays off on the AVR boards unless the bootloader is replaced; and nornWIFI needs the heap, event groups and 32 event bits.

4.3 The configuration signature

rtos_init is a macro that passes RTOS_CONFIG_SIGNATURE, every layout changing option folded into one 32 bit number, to the kernel. An application compiled against another rtos_config.h than the kernel objects it is linked with (a stale build directory, a library) panics with RTOS_PANIC_CONFIG at the first call, before any TCB is touched; the record carries both signatures.

4.4 A priority map

Priorities are unique and the module tasks need to be above every task that uses them. A map that works with the default configuration on the ESP32 (15 tasks): the nornWIFI radio task at 15 and its service task at 14, the nornNET net task at 13, the application's tasks from 1 to 12, with the most time critical highest. With the timer task enabled (TIMER_TASK_PRIO) its

slot must be left free by the application. Period and deadline, not importance, should set the order among the application's tasks: rate monotonic assignment is optimal for independent periodic tasks under fixed priorities [2], and the ceiling protocol keeps the blocking term small enough for the usual response time analysis [5] to apply.

Chapter 5

Using the primitives

nornCore offers more than one way to signal and to share. This chapter says what each primitive is for, how it behaves at the edges, and shows the idiom. The formal contracts are in chapter 7; the costs in chapter 9.

Need	Primitive	Cost, Mega / ESP32
wake one task, no data	task notification (`rtos_notify`) or event flags	220 / 145, 217 / 140
count occurrences, producer to consumer	counting semaphore, or `RTOS_NOTIFY_INC`	145 / 112
hand a small record to another task, in order	message queue	453 / 285
the latest value only, overwrite is fine	mailbox	282 / 194
flags several tasks wait on	event group	272 / 137
protect shared data between tasks	ceiling mutex	218 / 156
fixed size buffers, bounded time	memory pool	487 / 176
variable size, rarely, at start-up	heap	–
run something later or periodically without a task	software timer	241 / 124

Table 5.1: Which primitive for which need. The costs are cycles for one call pair without a switch (tc10, 2026-11-09).

5.1 Task event flags

Every task owns `RTOS_CFG_EVT_BITS` private event flags. Anyone posts (`rtos_postEvents`, `rtos_postEventsFromIsr`, `rtos_broadcastEvents`), only the owner waits. Events posted earlier stay pending until a wait consumes them, so nothing is lost between two waits; the wait returns the events that satisfied it and clears those, leaving other pending events alone.

```
#define EVT_SAMPLE  0x01u
#define EVT_BUTTON  0x02u

static void controller(void *arg)
{
    for(;;)
    {
        const rtos_evt_t got = rtos_waitEvents(EVT_SAMPLE | EVT_BUTTON,
    RTOS_WAIT_ANY, RTOS_MS(500));
        if(got == 0)            /* timeout: nothing in 500 ms */
            handleSilence();
        if(got & EVT_SAMPLE)
```

```
            processSample();
        if(got & EVT_BUTTON)
            handleButton();
    }
}

RTOS_ISR(ADC_vect)
{
    rtos_postEventsFromIsr(TASK_CONTROLLER, EVT_SAMPLE);
}
```

`RTOS_WAIT_ALL` waits for every bit of the mask. Event flags are the right tool when one task is the hub of several sources; they carry no data and do not count.

5.2 Counting semaphores

A semaphore counts tokens. `rtos_semGive` adds one, or hands it directly to the highest waiting task; `rtos_semTake` takes one or waits. `RTOS_SEM_INIT(n)` makes a semaphore without an upper limit, `RTOS_SEM_INIT_BINARY(n)` one whose count never exceeds one (a give while the token is available is dropped and counted in the `lost` field), `RTOS_SEM_INIT_MAX(n, max)` one with any limit. The typical use is an interrupt producing and a task consuming:

```
static rtos_sem_t _rxReady = RTOS_SEM_INIT(0);

RTOS_ISR(USART0_RX_vect)
{
    _rxBuf[_rxIn++ & RX_MASK] = UDR0;
    rtos_semGiveFromIsr(&_rxReady);
}

static void reader(void *arg)
{
    for(;;)
        if(rtos_semTake(&_rxReady, RTOS_FOREVER))
            consume(_rxBuf[_rxOut++ & RX_MASK]);
}
```

The semaphore here is only the count; the data travels in the application's own ring. A semaphore is not a lock: there is no owner and no priority ceiling, and using one for mutual exclusion brings back the priority inversion the mutex exists to prevent.

5.3 Ceiling mutexes

A mutex protects shared data between tasks. Its one parameter is the ceiling, the highest priority of any task that will lock it (section 3.6).

```
#define TASK_LOG     1
#define TASK_SENSOR  2
#define TASK_CTRL    4

static rtos_mutex_t _stateLock = RTOS_MUTEX_INIT(TASK_CTRL);   /* highest user */
static struct state _state;

static void sensor(void *arg)        /* priority 2 */
{
    for(;;)
```

```
    {
        (void)rtos_waitPeriodic(RTOS_MS(10));
        const int16_t t = readTemperature();
        rtos_mutexLock(&_stateLock);     /* runs at 4 until the unlock */
        _state.temperature = t;
        rtos_mutexUnlock(&_stateLock);
    }
}
```

While the sensor task holds the lock it runs at priority 4, so the controller (4) cannot preempt it and find the state half written, and the logger (1) is unaffected. The critical section is a few stores; keep it that way. Do not call anything that may block inside it.

5.4 Message queues

A queue is a FIFO of fixed size items copied by value, with blocking send and receive and non-blocking `FromIsr` variants. A send to a queue with a blocked receiver copies the item straight into the receiver's buffer. A full queue makes the sender wait or, with `RTOS_NO_WAIT` or from an interrupt, fail; failed sends are counted in the `dropped` field, and the high water mark in `maxCount`, so a queue sized too small is visible.

```
typedef struct { uint8_t channel; uint16_t value; } sample_t;

static uint8_t      _qStore[8 * sizeof(sample_t)];
static rtos_queue_t _q = RTOS_QUEUE_INIT(_qStore, sizeof(sample_t), 8);

RTOS_ISR(BOARD_APP_TIMER_VECT)
{
    sample_t s = { .channel = ADMUX & 0x0F, .value = ADC };
    (void)rtos_queueSendFromIsr(&_q, &s);        /* false if full: counted as
    dropped */
}

static void worker(void *arg)
{
    sample_t s;
    for(;;)
        if(rtos_queueReceive(&_q, &s, RTOS_MS(100)))
            process(&s);
        else
            noSampleFor100ms();
}
```

Items are copied twice (in and out), so a queue of large items is better replaced by a queue of pointers into a memory pool.

5.5 Mailboxes

A mailbox is a queue of depth one in which a post never blocks and overwrites an unread item (counted in `overwrites`). It carries *the latest value*: a setpoint, a measurement, a state that only the newest matters of. A reader blocks until something is posted. Use it where a queue would only fill with stale data.

5.6 Task notifications

The cheapest signal: one 16 bit value per task and no object. `rtos_notify` replaces, ORs into or increments the value and wakes the task if it waits; `rtos_notifyWait` takes the whole value, `rtos_notifyTake` waits for a non-zero value and decrements it, which with `RTOS_NOTIFY_INC` on the sending side is a counting semaphore without a semaphore. A task should use one of the two wait styles, since they share the value. Notifications are the fastest switch on both boards (figure 9.2).

```
RTOS_ISR(DMA_DONE_vect)
{
    rtos_notifyFromIsr(TASK_DMA, 1, RTOS_NOTIFY_INC);
}

static void dmaTask(void *arg)
{
    for(;;)
        if(rtos_notifyTake(RTOS_FOREVER))
            handleCompletion();
}
```

5.7 Software timers

A software timer calls a function after a delay, once or periodically, without a task of its own. With `RTOS_CFG_TIMER_TASK_PRIO 0` the callback runs in the tick interrupt with interrupts disabled: keep it short and use only the ISR body functions. With a timer task configured, the callback runs in that task and may use the whole API, at the price of a task slot and its stack.

```
static rtos_timer_t _blink;

static void blinkCb(void *arg)
{
    gpio_toggle(&_led);                  /* ISR body: no blocking, no switch */
}

/* in a task or in main() before rtos_start: */
rtos_timerInit(&_blink, blinkCb, NULL);
rtos_timerStart(&_blink, RTOS_MS(500), RTOS_MS(500));   /* first in 500 ms, then
    every 500 ms */
```

The first expiry is on the `delay`-th tick from now (so after $delay - 1$ to $delay$ periods); later expiries are exactly `period` ticks apart, counted from the previous expiry, so a periodic timer does not drift. A delay of 0 means the next tick, which lets a callback restart its own timer. The active timers are a list sorted by expiry; `rtos_timerStart` spends time proportional to their number with interrupts off.

5.8 Memory pools

A pool hands out fixed size blocks from storage the application owns, in constant time, with a wait for a free block if wanted and a `FromIsr` variant that does not wait. `rtos_poolFree` checks that the block belongs to the pool and lies on a block boundary. The low water mark `minFree` and the count of failed allocations `failed` show whether the pool is sized right. A pool is what a driver uses for packet buffers and what a queue of pointers refers to; it is the bounded time alternative to the heap.

```
static uint8_t     _frames[6 * 64];
static rtos_pool_t _framePool = RTOS_POOL_INIT(_frames, 64, 6);
/* rtos_poolInit(&_framePool, _frames, 64, 6) must still be called before use */

uint8_t *p = rtos_poolAlloc(&_framePool, RTOS_MS(10));
if(p != NULL) { fill(p); rtos_queueSend(&_frameQ, &p, RTOS_FOREVER); }
```

5.9 Event groups

An event group is a set of flags shared by any number of tasks (`RTOS_CFG_EVGROUPS`). Where task event flags belong to one task, a group lets several tasks wait for the same conditions: "link up and address assigned", say. `rtos_evgroupSet` wakes every waiter whose condition is now met, highest first; a waiter that asked for `RTOS_WAIT_CLEAR` takes its bits out of the group as it is woken, so a lower waiter no longer sees them. Without it the bits stay set for everyone until `rtos_evgroupClear`.

5.10 Suspend and resume

`rtos_suspend` keeps a task from running until `rtos_resume`, whatever it was doing. A blocked task stays in its wait; a wake-up that arrives meanwhile is kept and takes effect at the resume. There is no nesting count, and a suspended task keeps the mutexes it holds, which is the reason to suspend only tasks that are known to hold none. Suspension is for modes ("the display task is off while the lid is closed"), not for synchronisation; the primitives above are.

5.11 The heap

`RTOS_CFG_HEAP` compiles a first fit allocator with coalescing over an area the application gives it (`rtos_heapInit`). `rtos_malloc` searches the free list with interrupts disabled, so its time grows with fragmentation; it is meant for start-up allocations and for modules whose binary libraries need an allocator (nornWIFI), not for the data path. `rtos_heapStats` reports the low water mark and the largest free block.

Chapter 6

Robustness and diagnostics

A kernel for a device that runs unattended has to answer two questions: what does it do when the application breaks a rule, and how does the engineer find out what happened. The answers in nornCore are a small number of mandatory checks, a panic record that survives a reset, and a set of diagnostics that read the kernel's own bookkeeping.

6.1 Checks and assertions

Every public function that takes a task id, a storage pointer, a stack size or a timeout validates it with `RTOS_CHECK`, in every configuration. A violation is a *panic* with `RTOS_PANIC_ARGUMENT` and the two offending values in the record; there is no error return for an argument that would corrupt memory, because a return value is what the caller that made the mistake would ignore. Contracts whose violation costs more to detect than it is worth in a release build are `RTOS_ASSERT`ed instead and compiled only with `RTOS_CFG_ASSERT`: recursive locks, the unlock order, a lock from the idle task, an empty event mask. Chapter 7 marks every entry *checked* or *asserted*.

6.2 Panic and the record

`rtos_panic(reason, arg1, arg2)` disables interrupts, writes a record (reason, running task, tick, the two arguments, and for an assertion the file and line) into memory that survives a reset, calls `rtos_panicHook` and then halts (`RTOS_CFG_PANIC_ACTION 0`) or resets the board (1). After a reset, `rtos_init` takes the record out of that memory and `rtos_getLastPanic` returns it once; `rtos_getResetCause` says what the hardware thinks happened (power on, external, brown out, watchdog, software).

Reason	Meaning, `arg1` / `arg2`
`RTOS_PANIC_ASSERT`	a failed `RTOS_ASSERT`; file and line in the record
`RTOS_PANIC_STACK_OVERFLOW`	the canary of task `arg1` was overwritten (`STACK_CHECK 2`)
`RTOS_PANIC_WATCHDOG`	task `arg1` did not check in; `RTOS_NO_TASK`: the idle task
`RTOS_PANIC_DEADLOCK`	task `arg1` would wait for a mutex held, through a chain, by itself; `arg2` the owner
`RTOS_PANIC_FAULT`	a CPU exception; cause and address from the port
`RTOS_PANIC_CONFIG`	the application's and the kernel's configuration signatures differ
`RTOS_PANIC_ARGUMENT`	a mandatory check refused an argument; the values
`RTOS_PANIC_APPLICATION`	`rtos_panic` called by the application; its own arguments

Table 6.1: Panic reasons.

tc05 provokes four of these in turn on the boards (an application panic, a stack overflow,

a late task, a starved idle task), lets each reset the board, and checks the record after every reset.

6.3 Stack checking

Section 3.8 has the sizes; this is the machinery. With `RTOS_CFG_STACK_CHECK` the stack is filled with a pattern at creation and `rtos_getStackFree` counts the untouched bytes from the bottom: the high water mark since the start. With `RTOS_CFG_STACK_WARN` the idle task reads one task's mark per loop and calls `rtos_stackWarnHook` once for a task whose margin drops below `RTOS_PORT_STACK_WARN`. With `STACK_CHECK 2` a canary at the bottom is checked before every context switch and at every tick, and an overwritten canary is `RTOS_PANIC_STACK_OVERFLOW`. The canary catches an overflow after the fact; the monitor warns before it.

6.4 The watchdog

`RTOS_CFG_WATCHDOG` arms the hardware watchdog (`rtos_wdtInit`) and lets the idle task kick it, but only after checking that every monitored task (`rtos_wdtMonitor`) has called `rtos_wdtCheckIn` within its interval. A late task is `RTOS_PANIC_WATCHDOG` with a record; a starved idle task, which means a runaway task or an interrupt storm, lets the hardware reset the board without one, and the next boot sees `RTOS_RESET_WATCHDOG`. The two cases together cover both ways a system can hang.

6.5 Deadlock detection

A mutex wait that would close a cycle (A waits for a mutex B holds while B waits for one A holds, through any number of tasks) is detected at the moment of the wait and is `RTOS_PANIC_DEADLOCK`. With correct ceilings a wait never happens, so the detector is a safety net for a wrong ceiling or an owner that blocked while holding.

6.6 Run time statistics and task information

`RTOS_CFG_RUNTIME_STATS` measures the time each task runs, at every switch, in counts of `RTOS_RUNTIME_HZ`. `rtos_getTaskRunTime` returns a task's total, `rtos_getCpuLoad` the share of time not spent in the idle task since the previous call. `rtos_getTaskInfo` is the snapshot for a console or a debugger: state, priority and effective priority, what the task waits on and until when, stack size and free, run time, overruns. `rtos_getOverrunCount` reads and optionally resets a periodic task's overruns. The kernel objects also carry their own statistics: lost gives on a semaphore, dropped sends and the high water mark on a queue, overwrites on a mailbox, failed allocations and the low water mark on a pool, the heap's statistics.

6.7 Trace

`RTOS_CFG_TRACE` calls `rtos_traceHook(event, a, b)` at every context switch (the tasks leaving and entering), every block (task and wait kind), every wake-up, every timer expiry and every panic, with interrupts disabled. `hal/trace.h` is a ring buffer recorder to override the hook with, and the host unit tests are built on exactly this trace: each test compares the recorded sequence of (task, event, value) with the expected one.

Chapter 7

API reference

The reference follows the groups of `kernel/rtos.h`. Every entry gives the prototype, the behaviour, the parameters, the calling context, and which contracts are *checked* (always) or *asserted* (`RTOS_CFG_ASSERT`). Unless an entry says otherwise, a `timeout` is in ticks: `RTOS_NO_WAIT` polls, `RTOS_FOREVER` waits for good, any other value is at most `RTOS_MAX_TIMEOUT` (checked). Functions that may wake a higher task run it before they return; their `FromIsr` variants defer the switch to the end of the interrupt. The generated reference (`make api` in nornDocs) has the same text per function with the remaining details.

7.1 Constants and types

Name	Meaning
`RTOS_VERSION`	version string of the kernel
`RTOS_IDLE_TASK`	0, the idle task
`RTOS_NUM_TASKS`	`RTOS_CFG_NUM_TASKS + 1`
`RTOS_NO_TASK`	0xFF, “no task”
`RTOS_STACK_MIN / DEFAULT / LARGE`	the port’s stack sizes (table 3.3)
`RTOS_FOREVER`, `RTOS_NO_WAIT`	timeouts: never, poll
`RTOS_MAX_TIMEOUT`	the longest finite timeout, half the tick range
`RTOS_MS(ms)`	milliseconds to ticks, at compile time
`RTOS_WAIT_ANY / ALL`	options of `rtos_waitEvents`, `rtos_evgroupWait`
`RTOS_WAIT_CLEAR`	`rtos_evgroupWait`: take the satisfying bits out of the group
`RTOS_NOTIFY_SET / OR / INC`	operations of `rtos_notify`
`RTOS_SEM_INIT(n)`, `RTOS_SEM_INIT_BINARY(n)`, `RTOS_SEM_INIT_MAX(n, max)`	static semaphore initialisers
`RTOS_MUTEX_INIT(ceiling)`	static mutex initialiser
`RTOS_QUEUE_INIT(storage, itemSize, capacity)`	static queue initialiser
`RTOS_MBOX_INIT(storage, itemSize)`	static mailbox initialiser
`RTOS_POOL_INIT(storage, blockSize, numBlocks)`	static pool initialiser; `rtos_poolInit` is still required
`RTOS_EVGROUP_INIT(bits)`	static event group initialiser
`RTOS_RUNTIME_HZ`, `RTOS_RUNTIME_TO_MS(t)`	run time counter frequency and conversion
`rtos_tick_t`, `rtos_evt_t`, `rtos_taskmask_t`, `rtos_runtime_t`	the tick, event flag, task mask and run time types; widths from the configuration and the port
`rtos_taskFunc_t`, `rtos_timerFunc_t`	`void f(void *arg)`
`rtos_sem_t`, `rtos_mutex_t`, `rtos_queue_t`, `rtos_mbox_t`, `rtos_pool_t`, `rtos_timer_t`, `rtos_evgroup_t`	the kernel objects; treat as opaque except for the statistics fields named in chapter 6
`rtos_taskInfo_t`, `rtos_panicInfo_t`, `rtos_heapStats_t`	diagnostic records
`RTOS_TASK_*`, `RTOS_WAIT_*`, `RTOS_RESET_*`, `RTOS_TRACE_*`	task states, wait kinds, reset causes, trace events

Table 7.1: Constants and types.

7.2 Kernel life cycle

`rtos_init`

```
#define rtos_init()   rtos_initChecked(RTOS_CONFIG_SIGNATURE)
```

Resets all kernel data: every task slot, the ready mask, the wake-up list and the tick count, and takes the panic record of the previous boot out of the memory that survives a reset. The macro passes the configuration signature; a kernel compiled with another configuration stops at once with `RTOS_PANIC_CONFIG`.

Precondition: Called from `main()`, before any other kernel function, once.

rtos_createTask

```
void rtos_createTask(uint8_t prio, rtos_taskFunc_t taskFunc, void *arg,
                     uint8_t *stackArea, size_t stackSize);
```

Creates the task at priority `prio` on the caller's stack. The task is ready at once and runs at the earliest at `rtos_start`. The stack start is aligned up and the size rounded down to the port's alignment (1 byte on the AVR, 16 on the ESP32), which may reduce the usable size; with `RTOS_CFG_STACK_CHECK` the area is filled with the pattern.

prio task priority and id, 1 to `RTOS_CFG_NUM_TASKS`; the slot must be free.

taskFunc the task's function; should never return (section 3.7).

arg passed to the function unchanged.

stackArea, stackSize caller allocated memory, valid for as long as the system runs; at least `RTOS_STACK_MIN` after alignment.

Precondition: After `rtos_init`, before `rtos_start`. Creating a task while the scheduler runs is not supported yet.

Checked: `prio` out of range, a NULL function or stack, a slot in use, a stack below the minimum.

rtos_start

```
void rtos_start(void);
```

Creates the timer task if configured, starts the tick, switches to the highest ready task and never returns. The calling context becomes the idle task.

7.3 Time and delays

rtos_getTick

```
rtos_tick_t rtos_getTick(void);
```

Returns: the ticks since `rtos_start`. The counter wraps (16 bits: 65.5 s at 1 kHz; 32 bits: 49.7 days); compare as differences.

Context: any.

rtos_delay

```
void rtos_delay(rtos_tick_t ticks);
```

Blocks the calling task for at least `ticks` ticks; the real delay is between `ticks` and `ticks + 1` periods. A delay of 0 waits for the next tick. Any value is accepted: a delay above `RTOS_MAX_TIMEOUT` is done as a sequence of waits, `RTOS_FOREVER` blocks for good.

Context: task; not the idle task (checked).

rtos_waitPeriodic

```
bool rtos_waitPeriodic(rtos_tick_t period);
```

Blocks until the task's next period boundary; each boundary is the previous one plus `period`, so the task runs drift free. If the boundary has passed, the call returns at once, counts an overrun and computes the next boundary from now. The first boundary is the task's start plus `period`. The period may change from call to call.

period 1 to `RTOS_MAX_TIMEOUT`.

Returns: `true` on an overrun, `false` after a normal wait.
Context: task; not the idle task.
Checked: a period of 0 or above the limit.

7.4 Task event flags

rtos_waitEvents

```
rtos_evt_t rtos_waitEvents(rtos_evt_t mask, uint8_t opt, rtos_tick_t timeout);
```

Waits until events of `mask` are pending for the calling task. Events posted earlier stay pending until a wait consumes them. The events that satisfy the wait are cleared and returned; pending events outside the mask are kept.

mask the events to wait for; not 0 (asserted).

opt `RTOS_WAIT_ANY` or `RTOS_WAIT_ALL`.

Returns: the consumed events (with `RTOS_WAIT_ALL`: the mask); 0 on timeout.
Context: task; the idle task may only poll.

rtos_postEvents, rtos_postEventsFromIsr

```
void rtos_postEvents(uint8_t taskId, rtos_evt_t events);
void rtos_postEventsFromIsr(uint8_t taskId, rtos_evt_t events);
```

ORs the events into the task's pending events; a task whose wait is thereby satisfied becomes ready.
Checked: `taskId` 1 to `RTOS_CFG_NUM_TASKS`.

rtos_broadcastEvents

```
void rtos_broadcastEvents(rtos_evt_t events);
```

Posts the events to every created task (not the idle task).
Context: task.

7.5 Counting semaphores

rtos_semInit, rtos_semInitMax

```
void rtos_semInit(rtos_sem_t *pSem, uint8_t initialCount);
void rtos_semInitMax(rtos_sem_t *pSem, uint8_t initialCount, uint8_t maxCount);
```

The run time alternatives to the static initialisers. Without a limit the count is bounded by its type, 255; with `maxCount` a give at the limit is dropped and counted as lost (1 makes a binary semaphore).
Precondition: no task waits on the semaphore.
Asserted: `initialCount` at most `maxCount`, `maxCount` at least 1.

rtos_semTake

```
bool rtos_semTake(rtos_sem_t *pSem, rtos_tick_t timeout);
```

Takes one token, waiting up to `timeout` for it.
Returns: `true` if a token was taken, `false` on timeout.
Context: task; the idle task may only poll.

rtos_semGive, rtos_semGiveFromIsr

```
void rtos_semGive(rtos_sem_t *pSem);
void rtos_semGiveFromIsr(rtos_sem_t *pSem);
```

Gives one token. If tasks wait, the highest gets it directly and the count does not change; otherwise the count goes up or, at the limit, the give is lost.

7.6 Ceiling mutexes

rtos_mutexInit

```
void rtos_mutexInit(rtos_mutex_t *pMutex, uint8_t ceilingPrio);
```

The run time alternative to `RTOS_MUTEX_INIT`. `ceilingPrio` is the highest priority of any task that will ever lock the mutex.

Precondition: the mutex is not locked.

rtos_mutexLock

```
void rtos_mutexLock(rtos_mutex_t *pMutex);
```

Raises the caller's effective priority to the ceiling and takes the mutex. With a correct ceiling the call never blocks. If the mutex is held anyway (a ceiling too low, or an owner that blocked while holding), the caller waits without a timeout and the unlock hands the mutex over to the highest waiter.

Context: task; not the idle task (asserted).

Asserted: the caller does not already hold the mutex (not recursive); the caller's priority is at most the ceiling.

Panics: a wait that would close a cycle of mutex waits is `RTOS_PANIC_DEADLOCK` (the caller, the owner).

rtos_mutexUnlock

```
void rtos_mutexUnlock(rtos_mutex_t *pMutex);
```

Drops the caller's effective priority back to what it was before the lock. A waiting task becomes the owner at once, at the ceiling. Either change may let another task run before the call returns.

Context: task.

Asserted: the caller owns the mutex; mutexes are unlocked in the reverse order of locking.

7.7 Message queues

rtos_queueInit

```
void rtos_queueInit(rtos_queue_t *pQ, uint8_t *storage, uint8_t itemSize, uint8_t
    capacity);
```

The run time alternative to `RTOS_QUEUE_INIT`. `storage` holds `itemSize * capacity` bytes and stays valid for as long as the queue is used.

Checked: a NULL storage, an item size or a capacity of 0.

rtos_queueSend, rtos_queueSendFromIsr

```
bool rtos_queueSend(rtos_queue_t *pQ, const void *pItem, rtos_tick_t timeout);
bool rtos_queueSendFromIsr(rtos_queue_t *pQ, const void *pItem);
```

Copies an item into the queue, waiting up to `timeout` for room (the ISR variant never waits). If a task is blocked in receive, the item goes straight into that task's buffer. While the caller waits, the item is read at the moment a receiver frees a slot: it must not change until the call returns. A failed send is counted as dropped.

Returns: `true` if the item was queued.

rtos_queueReceive, rtos_queueReceiveFromIsr

```
bool rtos_queueReceive(rtos_queue_t *pQ, void *pItem, rtos_tick_t timeout);
bool rtos_queueReceiveFromIsr(rtos_queue_t *pQ, void *pItem);
```

Copies the oldest item out, waiting up to `timeout` for one. If a sender is blocked because the queue was full, its item moves into the freed slot.

Returns: `true` if an item was received.

rtos_queueCount

```
uint8_t rtos_queueCount(const rtos_queue_t *pQ);
```

Returns: the items stored now; a snapshot.

Context: any.

7.8 Mailboxes

rtos_mboxInit, rtos_mboxPost, rtos_mboxPostFromIsr, rtos_mboxFetch, rtos_mboxIsFul

```
void rtos_mboxInit(rtos_mbox_t *pMb, uint8_t *storage, uint8_t itemSize);
bool rtos_mboxPost(rtos_mbox_t *pMb, const void *pItem);
bool rtos_mboxPostFromIsr(rtos_mbox_t *pMb, const void *pItem);
bool rtos_mboxFetch(rtos_mbox_t *pMb, void *pItem, rtos_tick_t timeout);
bool rtos_mboxIsFull(const rtos_mbox_t *pMb);
```

A post never blocks: it replaces an unread item (counted in `overwrites`, and the post returns `true`) or, if a task is blocked in fetch, copies the item straight into that task's buffer. A fetch takes the item and leaves the mailbox empty; it returns `false` on timeout. `rtos_mboxIsFull` tells whether an unread item waits.

Checked: `rtos_mboxInit`: a NULL storage or an item size of 0.

7.9 Task notifications

rtos_notify, rtos_notifyFromIsr

```
void rtos_notify(uint8_t taskId, uint16_t value, uint8_t op);
void rtos_notifyFromIsr(uint8_t taskId, uint16_t value, uint8_t op);
```

Updates the task's 16 bit notification value and wakes the task if it waits in `rtos_notifyWait`, or in `rtos_notifyTake` and the value is now non-zero.

op `RTOS_NOTIFY_SET` replaces the value, `RTOS_NOTIFY_OR` ORs `value` in, `RTOS_NOTIFY_INC` adds one, saturating at 0xFFFF.

Checked: `taskId`.

rtos_notifyWait

```
bool rtos_notifyWait(uint16_t *pValue, rtos_tick_t timeout);
```

Returns as soon as any notification has arrived since the last successful wait, even one that set the value to 0, stores the value in `*pValue` (may be NULL) and resets it to 0.

Context: task; the idle task may only poll.

rtos_notifyTake

```
bool rtos_notifyTake(rtos_tick_t timeout);
```

Waits until the value is non-zero and decrements it by one: with `RTOS_NOTIFY_INC` on the sending side, a counting semaphore without an object. A task should use either `rtos_notifyWait` or `rtos_notifyTake`, not both.

7.10 Software timers

rtos_timerInit

```
void rtos_timerInit(rtos_timer_t *pT, rtos_timerFunc_t callback, void *arg);
```

Prepares an inactive timer. Where the callback runs depends on `RTOS_CFG_TIMER_TASK_PRIO`: in the tick interrupt (0; only ISR body functions allowed) or in the timer task.

Checked: a NULL callback.

rtos_timerStart

```
void rtos_timerStart(rtos_timer_t *pT, rtos_tick_t delay, rtos_tick_t period);
```

Arms the timer, or restarts it if active. The first expiry is on the `delay`-th tick from now (after `delay - 1` to `delay` periods; 0 and 1 both mean the next tick); after that every `period` ticks exactly, counted from the previous expiry. `period` 0 is a one shot timer. The time spent with interrupts disabled grows with the number of active timers.

Context: any.

Checked: the timer was initialised; `delay` and `period` at most the limit.

rtos_timerStop, rtos_timerIsActive, rtos_timerRemaining

```
void        rtos_timerStop(rtos_timer_t *pT);
bool        rtos_timerIsActive(const rtos_timer_t *pT);
rtos_tick_t rtos_timerRemaining(const rtos_timer_t *pT);
```

Disarms the timer (nothing happens if it is not active; with a timer task an expiry still waiting for the task is dropped, a running callback finishes); tells whether it is armed; returns the ticks to the next expiry (0 if inactive or due).

Context: any.

7.11 Memory pools

rtos_poolInit

```
void rtos_poolInit(rtos_pool_t *pPool, uint8_t *storage, uint16_t blockSize,
    uint8_t numBlocks);
```

Builds the free list over the storage; also required after `RTOS_POOL_INIT`. A block is returned at the alignment of its offset in the storage, so storage meant for aligned objects must be aligned itself and have a block size that is a multiple of that alignment.
Checked: a NULL storage, a block size below `sizeof(void *)`, 0 blocks.

rtos_poolAlloc, rtos_poolAllocFromIsr

```
void *rtos_poolAlloc(rtos_pool_t *pPool, rtos_tick_t timeout);
void *rtos_poolAllocFromIsr(rtos_pool_t *pPool);
```

Gets a block, waiting up to `timeout` for one to be freed (the ISR variant never waits). A failed allocation is counted.
Returns: the block, or NULL.

rtos_poolFree, rtos_poolFreeFromIsr, rtos_poolFreeCount

```
void    rtos_poolFree(rtos_pool_t *pPool, void *pBlock);
void    rtos_poolFreeFromIsr(rtos_pool_t *pPool, void *pBlock);
uint8_t rtos_poolFreeCount(const rtos_pool_t *pPool);
```

Returns a block; a task waiting for one gets it directly.
Checked: a pointer outside the pool or not at the start of a block. A double free is caught only when the pool would hold more free blocks than it has.

7.12 Event groups

rtos_evgroupInit, rtos_evgroupSet, rtos_evgroupSetFromIsr, rtos_evgroupClear, rtos_evgroupGet

```
void       rtos_evgroupInit(rtos_evgroup_t *pG, rtos_evt_t initialBits);
void       rtos_evgroupSet(rtos_evgroup_t *pG, rtos_evt_t bits);
void       rtos_evgroupSetFromIsr(rtos_evgroup_t *pG, rtos_evt_t bits);
void       rtos_evgroupClear(rtos_evgroup_t *pG, rtos_evt_t bits);
rtos_evt_t rtos_evgroupGet(const rtos_evgroup_t *pG);
```

Set wakes every waiter whose condition is now met, highest priority first; a waiter that asked for `RTOS_WAIT_CLEAR` takes its bits out as it is woken. Clear wakes nobody. With waiters present, set checks every task slot with interrupts disabled.

rtos_evgroupWait

```
rtos_evt_t rtos_evgroupWait(rtos_evgroup_t *pG, rtos_evt_t mask, uint8_t options,
                            rtos_tick_t timeout);
```

Waits until bits of `mask` are set. `options` is `RTOS_WAIT_ANY` or `RTOS_WAIT_ALL`, optionally with `RTOS_WAIT_CLEAR`.
Returns: the satisfying bits (with ALL: the mask); 0 on timeout.
Asserted: `mask` not 0.

7.13 Suspend and resume

rtos_suspend, rtos_resume, rtos_resumeFromIsr

```
void rtos_suspend(uint8_t taskId);
void rtos_resume(uint8_t taskId);
void rtos_resumeFromIsr(uint8_t taskId);
```

Suspend keeps the task from running until a resume; a blocked task stays in its wait, a wake-up that arrives meanwhile takes effect at the resume. No nesting count; a task may suspend itself. A suspended task keeps its mutexes. Resume does nothing to a task that is not suspended.

Checked: `taskId`; not the idle task.

7.14 Heap

rtos_heapInit, rtos_malloc, rtos_free, rtos_heapStats

```
void  rtos_heapInit(void *area, size_t size);
void *rtos_malloc(size_t size);
void  rtos_free(void *p);
void  rtos_heapStats(rtos_heapStats_t *pStats);
```

A first fit allocator with coalescing over the area given once to `rtos_heapInit`. Blocks are aligned to `2 * sizeof(void *)`. The search runs with interrupts disabled and its time grows with the number of free blocks. `rtos_free` ignores NULL.

Context: task.

Checked: a NULL or too small area; `rtos_malloc` before `rtos_heapInit`; a freed pointer outside the heap or already free. A pointer into the middle of a block is not detected.

7.15 Diagnostics

rtos_getCurrentTask

```
uint8_t rtos_getCurrentTask(void);
```

Returns: the running task's id; 0 in the idle task; in an ISR body the interrupted task.

rtos_getOverrunCount

```
uint8_t rtos_getOverrunCount(uint8_t taskId, bool reset);
```

Returns: the period overruns of the task since the start or the last reset; saturates at 255. `reset` clears the counter after reading.

Checked: `taskId` 0 to `RTOS_CFG_NUM_TASKS`.

rtos_getStackFree

```
size_t rtos_getStackFree(uint8_t taskId);
```

Returns: the bytes at the bottom of the task's stack never used since creation; 0 for the idle task, a task never created, or without `RTOS_CFG_STACK_CHECK`. The scan reads the stack byte by byte with interrupts enabled.

rtos_getTimeUs

```
uint32_t rtos_getTimeUs(void);
```

Returns: microseconds since start from the port's free running time base; wraps after 71.6 minutes.
Context: any.

rtos_getTaskRunTime, rtos_getCpuLoad

```
rtos_runtime_t rtos_getTaskRunTime(uint8_t taskId);
uint8_t        rtos_getCpuLoad(void);
```

With RTOS_CFG_RUNTIME_STATS: the accumulated run time of a task in counts of RTOS_RUNTIME_HZ (the running task's includes the current slice), and the share of time not spent in the idle task since the previous call, 0 to 100. Every call of rtos_getCpuLoad starts a new interval, so one task should own the readings.

rtos_getTaskInfo

```
bool rtos_getTaskInfo(uint8_t taskId, rtos_taskInfo_t *pInfo);
```

Fills a snapshot: state (RTOS_TASK_*), priority, effective priority, wait kind and object, timeout, stack size and free, run time, overruns.
Returns: true; false for an id out of range or a NULL pInfo (not a panic).

7.16 Panic and reset cause

rtos_panic

```
void rtos_panic(uint8_t reason, uint32_t arg1, uint32_t arg2) __attribute__((
    noreturn));
```

Disables interrupts, records the reason and the arguments in memory that survives a reset, calls rtos_panicHook and halts or resets (RTOS_CFG_PANIC_ACTION). Applications use RTOS_PANIC_APPLICATION.
Context: any.

rtos_getLastPanic, rtos_getResetCause

```
bool    rtos_getLastPanic(rtos_panicInfo_t *pInfo);
uint8_t rtos_getResetCause(void);
```

The record left before the last reset, reported in one boot only (pInfo may be NULL to ask only), and the cause of the last reset as the hardware reports it (RTOS_RESET_*).

7.17 Hooks

rtos_idleHook, rtos_panicHook, rtos_stackWarnHook, rtos_assertFailed, rtos_traceH

```
void rtos_idleHook(void);
void rtos_panicHook(const rtos_panicInfo_t *pInfo);
void rtos_stackWarnHook(uint8_t task, size_t stackFree);
void rtos_assertFailed(const char *file, uint16_t line) __attribute__((noreturn,
    cold));
void rtos_traceHook(uint8_t event, uint32_t a, uint32_t b);
```

Weak functions the application overrides. The idle hook runs in every idle loop and must not block. The panic hook runs with interrupts disabled before the halt or reset and must not call the kernel. The stack warning hook is called once per task whose margin is gone. `rtos_assertFailed` is called for a failed assertion; its default stores file and line and panics, and an override must not return either. The trace hook is called with interrupts disabled at every traced event (`RTOS_TRACE_SWITCH`, `BLOCK`, `READY`, `TIMER`, `PANIC`) and must not call the kernel.

7.18 Watchdog

`rtos_wdtInit, rtos_wdtMonitor, rtos_wdtCheckIn`

```
void rtos_wdtInit(uint32_t timeoutMs);
void rtos_wdtMonitor(uint8_t task, rtos_tick_t maxInterval);
void rtos_wdtCheckIn(void);
```

Starts the hardware watchdog (the timeout is rounded up to what the hardware offers; the idle task kicks it only after `rtos_start`); requires a task to check in at least every `maxInterval` ticks (0 stops monitoring it; the first deadline is `maxInterval` from now); tells the watchdog the calling task is alive.

Checked: `task`; `maxInterval` at most the limit.

Asserted: `timeoutMs` above 0.

Chapter 8

Networking: nornNET, nornWIFI and nornHTTP

The network modules are separate repositories with their own READMEs and plans; this chapter is what an application writer needs to use them from nornRTOS. The design sketch, protocol scope and test record are in `nornNET/doc/plan.md`, `nornWIFI/doc/plan.md` and `nornHTTP/docs/` (contracts, measurements, plan).

8.1 nornNET

nornNET is a small TCP/IP stack: Ethernet, ARP, IPv4, ICMP, UDP, TCP (RFC 9293 with slow start, fast retransmit, delayed ACK, Nagle, zero window probing and out of order storage), a DHCP client and a DNS resolver, loosely modelled on microps [6] in structure. Packet buffers are fixed size blocks from a pool, owned by whoever holds the pointer; there is one net task, one re-entrant lock, and no heap in the data path. It is verified against the Linux kernel as the peer, over a TAP device, QEMU's user mode network and a serial line (SLIP) on the ESP32 board, and it runs on the Mega 2560 over SLIP at the line rate.

8.1.1 Attaching it to an application

In `app.mk`:

```
NORNNET_DIR = /path/to/nornNET
MODULES    += $(NORNNET_DIR)/nornnet.mk
```

and an `nn_config.h` next to the application with at least `NN_CFG_TASK_PRIO`, the net task's priority, above every task that uses the stack. Every `NN_CFG_*` option and its default is in `nn/nn_cfg.h`. In `main`, after `rtos_init`: `nn_init()`, register the devices, add the interfaces with `nn_ifaceAdd` (with an address, or empty and `nn_dhcpStart` from a task), then `rtos_start()`.

8.1.2 Sockets

Sockets are caller owned objects; for TCP the caller also owns the send and receive rings. Each call is either blocking with a timeout, from a task, or a callback that runs in the net task. The lock is re-entrant for the net task, so a callback may call the socket API.

```
nn_err_t nn_udpOpen(nn_udp_t *pSock);
nn_err_t nn_udpBind(nn_udp_t *pSock, nn_ip4_t addr, uint16_t port);
nn_err_t nn_udpConnect(nn_udp_t *pSock, nn_ip4_t addr, uint16_t port);
nn_err_t nn_udpSendTo(nn_udp_t *pSock, const void *p, size_t n, nn_ip4_t to,
    uint16_t port);
```

```
nn_err_t nn_udpSend(nn_udp_t *pSock, const void *p, size_t n);
int32_t  nn_udpRecvFrom(nn_udp_t *pSock, void *p, size_t n, nn_ip4_t *pFrom,
                        uint16_t *pPort, nn_ms_t timeout);
void     nn_udpOnReadable(nn_udp_t *pSock, nn_udpReadableFunc_t fn, void *arg);
nn_err_t nn_udpClose(nn_udp_t *pSock);

nn_err_t nn_tcpOpen(nn_tcp_t *pSock, void *pRxRing, uint16_t rxSize, void *pTxRing
    , uint16_t txSize);
nn_err_t nn_tcpBind(nn_tcp_t *pSock, nn_ip4_t addr, uint16_t port);
nn_err_t nn_tcpListen(nn_tcp_t *pSock);
nn_err_t nn_tcpAccept(nn_tcp_t *pListen, nn_tcp_t *pNew, nn_ms_t timeout);
nn_err_t nn_tcpConnect(nn_tcp_t *pSock, nn_ip4_t addr, uint16_t port, nn_ms_t
    timeout);
int32_t  nn_tcpSend(nn_tcp_t *pSock, const void *p, size_t n, nn_ms_t timeout);
int32_t  nn_tcpRecv(nn_tcp_t *pSock, void *p, size_t n, nn_ms_t timeout);
nn_err_t nn_tcpClose(nn_tcp_t *pSock);
nn_err_t nn_tcpAbort(nn_tcp_t *pSock);
void     nn_tcpOnEvent(nn_tcp_t *pSock, nn_tcpEventFunc_t fn, void *arg);

nn_err_t nn_dhcpStart(nn_dhcp_t *pD, nn_iface_t *pIf, nn_dhcpChangeFunc_t fn, void
     *arg);
nn_err_t nn_dnsResolve(const char *name, nn_ip4_t *pAddr, nn_ms_t timeout);
```

Errors are negative `nn_err_t` values: `NN_ERR_NOMEM` (no buffer), `NN_ERR_TIMEOUT`, `NN_ERR_DOWN` (the device or the address is gone), `NN_ERR_RESET`, `NN_ERR_REFUSED`, `NN_ERR_CLOSED`, `NN_ERR_PENDING` (a non-blocking call accepted) among them. Timeouts in nornNET are milliseconds (`nn_ms_t`).

8.1.3 Links that come and go

A driver whose carrier can disappear (WiFi, a cable) reports it with `nn_netdevLinkChanged(dev, up)` from any context. On a loss the net task drops the device's ARP entries; on a return every DHCP client on the device confirms its lease at once, keeping the address meanwhile, so an outage on the same network does not disturb TCP connections. When an interface's address changes or goes, the TCP connections bound to the old one end with `NN_ERR_DOWN`.

8.2 nornWIFI

Espressif ships the ESP32 radio as binary libraries written against a table of operating system functions. nornWIFI implements that table over the nornCore primitives, adds the chip glue (power domain, clocks, PHY calibration, interrupts), the WPA supplicant for WPA2 personal, and a nornNET device. The result is a station that scans, joins, gets an address by DHCP and carries TCP at 400 to 650 kB/s each way, with nornCore as the only kernel in the image.

8.2.1 Using it

`app.mk` adds both modules; `rtos_config.h` needs `RTOS_CFG_HEAP 1`, `RTOS_CFG_EVT_BITS 32`, `RTOS_CFG_EVGROUPS 1` and two spare task ids at the top (`NW_CFG_TASK_WIFI`, `NW_CFG_TASK_SERVICE`; by default the two highest). Then:

```
nw_init(NW_HEAP_SPARE_START, NW_HEAP_SPARE_SIZE);    /* in main, before rtos_start
    */
...
nw_start();                                            /* in a task */
nn_ifaceAdd(nw_netdev(), &iface, zero, zero, zero);
nw_connect("ssid", "password", 20000);                 /* or nw_connectStart +
    events */
```

```
nn_dhcpStart(&dhcp, &iface, onDhcp, NULL);
```

From then on the station keeps the link up by itself (`NW_CFG_RECONNECT`: straight back after a loss, then a delay doubling from 1 to 60 seconds) until `nw_disconnect`, and nornNET follows the link as described above. `nw_linkState`, `nw_rssi`, `nw_isConnected` and the events delivered through `nw_onEvent` (run in the service task, which must not call nornNET) tell what happens. Measured on the board: three forced drops recover in about 160 ms, with DHCP confirmed within 100 ms and a TCP connection intact.

```
void      nw_init(void *heap, size_t heapSize);
esp_err_t nw_start(void);
esp_err_t nw_scan(wifi_ap_record_t *pRecords, uint16_t *pCount, uint32_t timeoutMs
    );
esp_err_t nw_connect(const char *ssid, const char *password, uint32_t timeoutMs);
esp_err_t nw_connectStart(const char *ssid, const char *password);
esp_err_t nw_disconnect(void);
bool      nw_isConnected(void);
uint8_t   nw_linkState(void);
int8_t    nw_rssi(void);
struct nn_netdev *nw_netdev(void);
void      nw_onEvent(nw_eventFunc_t cb, void *arg);
void      nw_getStats(nw_stats_t *pStats);
```

The radio libraries are Espressif's and keep their Apache 2.0 licence; the supplicant is the upstream wpa_supplicant as ported by Espressif (BSD). Both are listed in the repository's `THIRD_PARTY_LICENSES.md`.

8.3 nornHTTP

nornHTTP is an HTTP/1.1 client in portable C99 with nothing underneath it: no heap, no global state, no recursion, no dependency on nornCore or nornNET. The application owns every buffer and every context, so they can be static, and the library talks to the network through three functions it is handed (send, receive, a millisecond clock). An adapter for a connected nornNET TCP socket is part of the module; the same code runs on Linux against the module's test server, which is where most of its verification happens. It does GET, POST, PUT, DELETE, PATCH, HEAD and OPTIONS, custom headers, request bodies from a buffer or streamed from a callback with a known length, Content-Length, chunked and close-delimited responses, trailers, informational responses, closed byte ranges with 206 checking, HTTP/1.0 servers, and keep-alive. It does not do TLS, redirects, compression, cookies, DNS or HTTP/2; TLS is a transport the application would supply.

The response body is streamed: the parser hands each span of decoded bytes to a callback as it arrives, so a download of megabytes needs only the receive buffer the application chose (the ESP32 example below uses 512 bytes). The size of a response does not decide the RAM cost; the longest header line and the longest request head do.

8.3.1 Using it

`app.mk` adds the sources and the adapter; the `wifi_http` example in nornRTOS has the lines. With a socket already connected (`nn_tcpConnect`), one request is:

```
static char line[256], request[256];          /* longest header line, request head
    */
static uint8_t receive[512], body[128];       /* one recv at a time, a small body
    */
static norn_http http;
```

```
norn_http_net_context net = { &socket, now_ms, NULL };   /* now_ms wraps nn_os_now
    */
norn_http_transport transport;
norn_http_config config = {0};
norn_http_body_buffer sink = { body, sizeof(body), 0 };
norn_http_request req;
norn_http_response resp;

norn_http_net_transport_init(&transport, &net);
norn_http_parser_config_init(&config.parser, line, sizeof(line));
config.parser.on_body = norn_http_buffer_body;    /* or your own callback */
config.parser.callback_context = &sink;
config.receive_buffer = receive;   config.receive_capacity = sizeof(receive);
config.request_buffer = request;   config.request_capacity = sizeof(request);
config.timeout_ms = 5000;                         /* the whole request and
    response */
norn_http_init(&http, &config, &transport);

norn_http_request_init(&req, "GET", "/hello", "192.168.1.10:18080", NULL, 0);
if(norn_http_execute(&http, &req, &resp) == NORN_HTTP_OK)
    ...                          /* resp.status, sink.length bytes in body, resp.
    reusable */
```

A POST adds `norn_http_add_header(&req, "Content-Type", "application/json")` and `norn_http_set_body(&req, json, length)`, or `norn_http_set_body_source` with a callback that fills the request buffer span by span when the body is not in RAM as a whole. A large download replaces the body callback with one that writes each span to its destination, or checks it, and stores nothing. `resp.reusable` says whether the same socket may carry the next request; `req.close` asks the server to close after this one.

Every function returns a `norn_http_result`; zero is success, the HTTP status is a separate matter in `resp.status`. After a negative result from `norn_http_execute` the connection is in an unknown state: abort the socket and call `norn_http_init` again (the context answers `NORN_HTTP_STALE` until then). An argument error (`INVALID`, or `LIMIT` from the request builder) is reported before anything is sent and leaves the connection as it was. The full contract, including what the parser rejects and why, is `nornHTTP/docs/contracts.md`; the API reference is generated from `norn_http.h` together with the kernel's.

8.3.2 What it costs

Measured on the NodeMCU-32S with the `wifi_http` example (GET, JSON POST, a 64 KiB download through the 512-byte buffer with every byte checked, keep-alive, then Connection: close): the whole image grows by 7.0 KB of flash, of which about 5.3 KB is the library and 0.2 KB the nornNET adapter. The library has no static RAM; the context is 184 bytes, a request 64 and a response 56, and the example's buffers add 1.2 KB. The main task's stack high-water went from 1.3 KB after the Wi-Fi join and TCP connect to at most 1.5 KB after the four requests, of 4 KB; the library's own frames are small (the deepest chain inside it is about 240 bytes), the rest is the TCP path and the kernel running on the caller's stack. Details and the method are in `nornHTTP/docs/measurements.md`.

Chapter 9

Performance

Every figure in this chapter was produced by the test applications tc10, tc11 and tc12 of nornCore, run by `tools/measure.sh` on the Arduino Mega 2560 (16 MHz) and the ESP32 board (240 MHz) and written to `doc/measurements/step10-lifecycle-after.md` on 2026-11-09. The graphs are generated from that summary; a new measurement regenerates them (`make data` in nornDocs). The same script records the ESP32 in QEMU, which is not cycle accurate and is omitted here except where stack use is concerned, for which it is exact.

9.1 Method

tc10 measures the kernel in CPU cycles with a cycle counter (a free running timer at F_{CPU} on the Mega, `CCOUNT` on the ESP32) and a probe interrupt whose period is dithered so that it lands at every phase of the tick and of the other activity. Everything in the test runs at once: the switch benchmark, the probe at about 770 Hz, the tick, and the default configuration (stack fill, run time statistics, assertions, software timers). Three kinds of figures come out:

- **API cost**: one call pair (give and take, lock and unlock, send and receive) on a single task with nobody waiting, so no switch happens. The minimum of 64 repetitions: the cost of the critical section and the bookkeeping.
- **Switch**: a low task signals a high one, which blocks again at once. Per switch, including the signalling call, minimum and average over 500 round trips. This is the number that matters for a design: how much a wake-up costs.
- **Interrupt**: the entry latency from the hardware event to the first instruction of the `RTOS_ISR` body, and the latency from a give in the ISR body to the first instruction of the woken task ("ISR to task"), minimum, average and maximum over a thousand probes landing at random phases.

simavr is cycle exact and gives the same numbers as the Mega board (the ISR to task minimum is one lucky probe and may differ by a few cycles between the two), so a simulator run of tc10 is the arbiter for an A/B comparison of two kernel variants.

tc11 measures the stack a task needs for the kernel's and the port's sake: tasks that do one thing each (only block, be woken from an interrupt, be interrupted while running, run the benchmark script) report their high water marks. **tc12** repeats this at the bottom of a call chain 0, 4, 8, 16 and 32 frames deep, in four phases.

9.2 The cost of a call

Figure 9.1 ranks the primitives. The semaphore and the event flags are the cheapest signals, the notification follows, the mutex costs a priority change on top of the bookkeeping, the

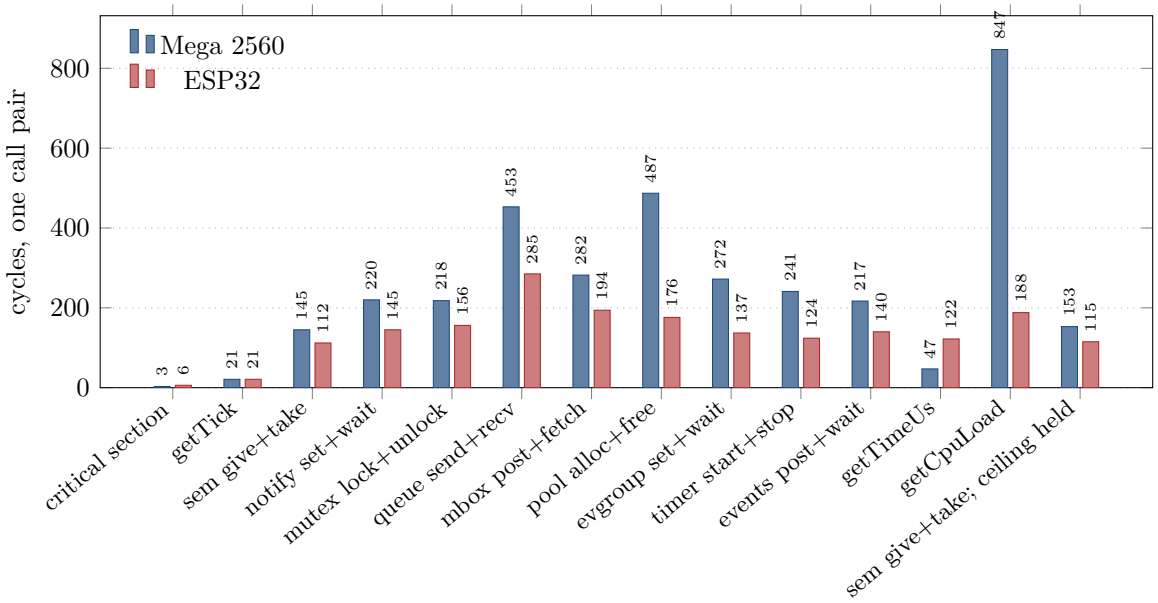


Figure 9.1: API cost in cycles for one call pair without a switch (tc10, 2026-11-09). The AVR is an 8 bit machine; its cycle counts are higher but its cycle is 15 times longer, so a semaphore pair costs 9 μs on the Mega and 0.47 μs on the ESP32. `getStackFree` (1694 / 6438 cycles, the scan of a whole stack) is left out of the graph.

queue copies the item twice, and the pool pays for the pointer alignment through `memcpy` on the AVR. `getCpuLoad` on the AVR is a 32 bit division; `getTimeUs` on the ESP32 is a division of the cycle counter. “Ceiling held” repeats the semaphore pair while the running task holds a mutex with a ceiling above its priority: the scheduler then has to consider the boosted mask, and the difference (8 cycles on the Mega, 3 on the ESP32) is the whole price of the ceiling protocol on the hot path.

9.3 The context switch

Where the cycles go on the AVR, per switch: about 105 in the context switch itself (19 registers each way), about 45 in the run time statistics, about 20 in the assertions, the rest in the scheduler, the wait list and the primitive. With statistics and assertions off the switch is 322 cycles (20 μs); table 4.2 has the three configurations. On the ESP32 a switch spills every live register window and the blocking path costs what a whole AVR stack does; with statistics off it is 361 cycles, with assertions off as well 351 (1.46 μs).

9.4 Interrupt latency

The AVR entry maximum of 34 μs and the ISR to task maximum of 52 μs are the same phenomenon: a 27 μs switch with interrupts off that the probe lands behind, plus a tick with a wake-up. Interrupts do not nest, so these maxima are also the worst case for any other interrupt in the system.

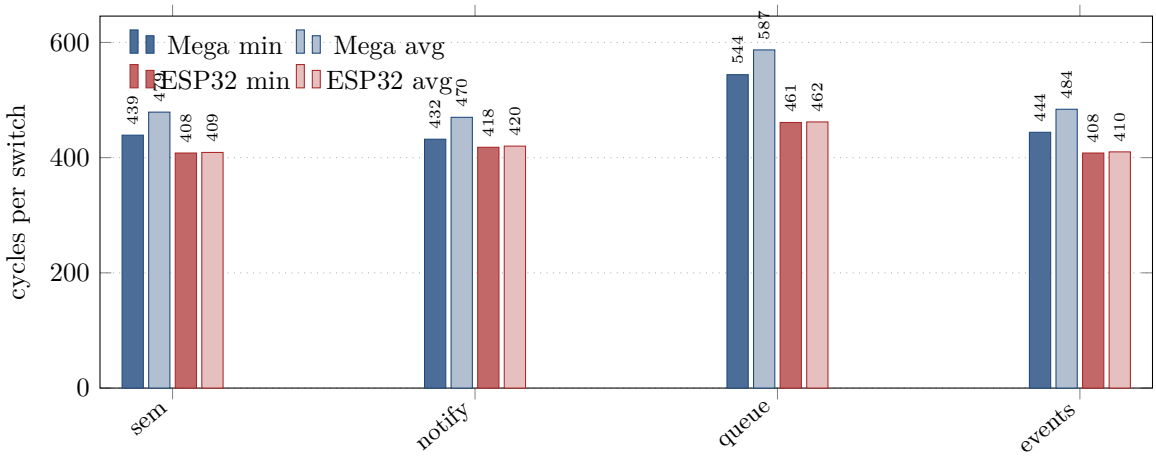


Figure 9.2: Cycles per context switch including the signalling call, by mechanism (tc10, 2026-11-09): 27.4 μs on the Mega and 1.70 μs on the ESP32 for the semaphore. The average exceeds the minimum on the Mega because the tick and the probe interrupt land inside some of the 500 round trips.

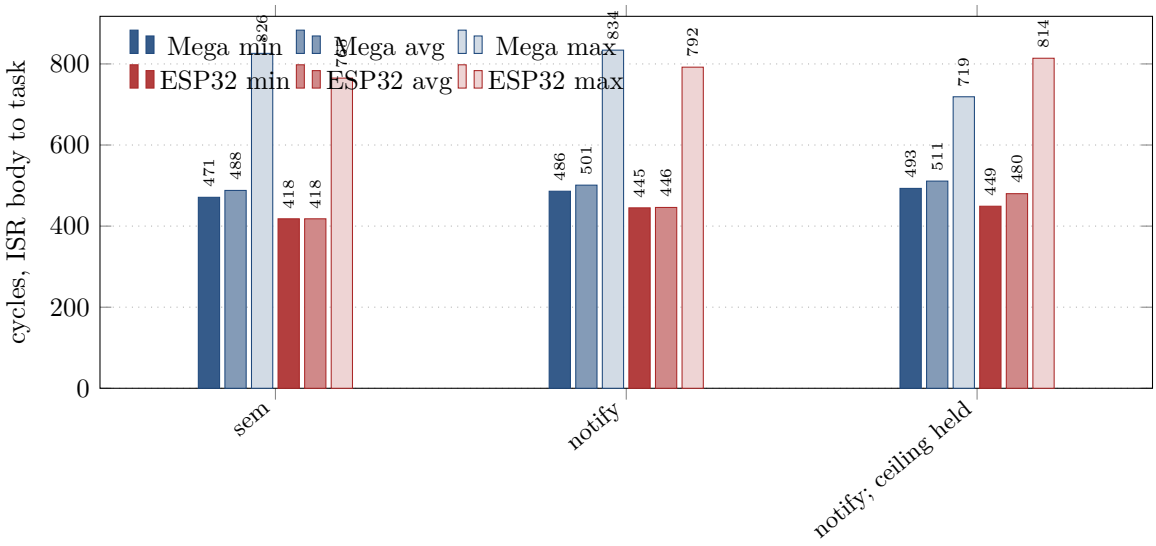


Figure 9.3: Latency from a give in the ISR body to the first instruction of the woken task, minimum, average and maximum over 1000 probes at random phases (tc10, 2026-11-09). The maximum is the longest interrupt-off window: a context switch that the probe lands behind.

	Mega 2560		ESP32	
	cycles	time	cycles	time
interrupt entry, min / avg / max	50 / 65 / 548	3.1 / 4.1 / 34 μs	– / 35 / 106	– / 0.9 / 2.7 μs
ISR to task (semaphore), min / max	471 / 826	29.4 / 51.6 μs	418 / 765	1.74 / 3.19 μs
hardware event to task, best case		33.5 μs		1.85 μs

Table 9.1: Interrupt figures (tc10, 2026-11-09). The ESP32 entry minimum reads 0 because the probe timer's 40 MHz counter is latched at the alarm. The entry cost on the ESP32 is the same 850 ns whether the interrupted task is 0 or 32 call frames deep (tc12): the window spill at entry depends on the live windows, at most about seven, not on the depth.

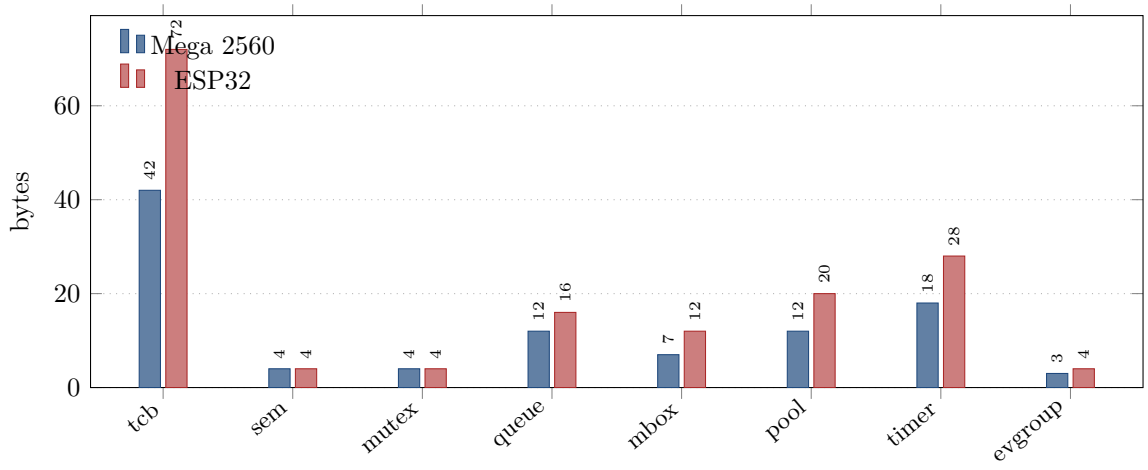


Figure 9.4: RAM per kernel object in the default configuration (tc10, 2026-11-09). The TCB is per task; the pointer width (2 and 4 bytes) explains most of the difference.

	image (tc01)			every kernel and port function		
Board	text	data	bss	text	data	bss
uno	6852	170	833	9909	8	240
mega2560	6998	170	833	9935	8	240
esp32	10138	24	3920	12975	13	815

Table 9.2: Code and data size in bytes (step10-lifecycle-after). The image is tc01 with its default configuration, after the linker dropped what it does not call; the right half is every kernel and port object compiled, the upper bound an application can pull in. The smallest running application, one task blinking the LED with everything off, is 2514 bytes of text on the UNO and 6089 on the ESP32.

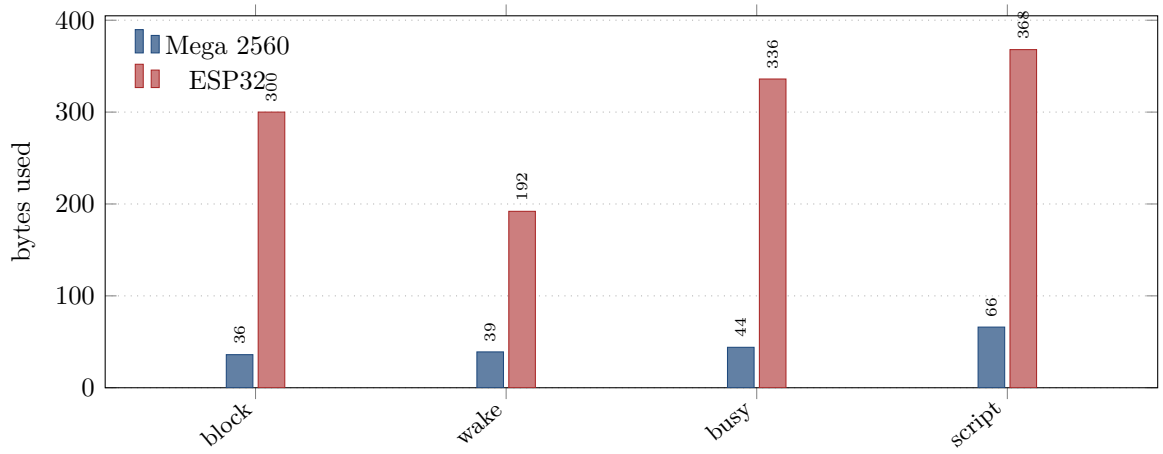


Figure 9.5: Stack used by tasks that do one thing each (tc11, 2026-11-09): *block* only blocks, *wake* is woken from an interrupt, *busy* is interrupted while running, *script* runs the test's own sequence. The ESP32 "block" figure is 192 when no interrupt lands on the task during its short runs and 300 when one does: size a stack by the interrupted figure, which does not depend on luck.

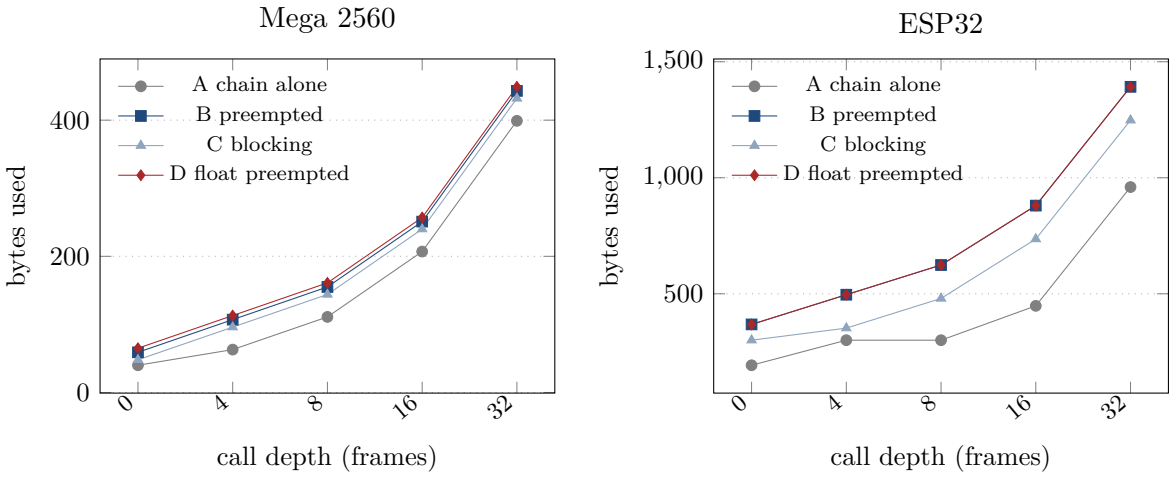


Figure 9.6: Stack used against call depth (tc12, 2026-11-09): the chain alone, preempted at the bottom with a switch out of the interrupt, blocking at the bottom, and floating point at the bottom. The lines are parallel: the kernel's and the port's share is constant (Mega 59 / 48 / 65 bytes, ESP32 368 / 300 / 368 including the task's two bottom frames), so the rule for a stack is the task's own frames (12 bytes per level on the Mega with a few locals, 32 per `call8` level on the ESP32) plus the "interrupted" figure.

9.5 Memory

9.6 Stacks

9.7 The cost of the configuration

Table 4.2 gives the switch for the three configurations: on the Mega the run time statistics cost about 45 cycles per switch and the assertions about 20; on the ESP32 the lean kernel saves 93 cycles over the default. `RTOS_CFG_STACK_CHECK` costs nothing on the hot path (the fill is at creation, the canary check a compare per switch), the software timers one list check per tick, and the ceiling protocol 8 cycles on the Mega only while a boosting mutex is held.

9.8 History

nornCore measures itself before and after every change to the kernel. Figure 9.7 shows the hot path and the code size across the steps since the baseline of 2026-11-05: the fixes of an independent audit (F1 to F11), the scheduler contract for mutex ceilings (tie), the exclusion of the running task's own boost from the slow path test (boost) and the task life cycle (lifecycle). Correctness bought about 13 cycles per switch on the Mega (F2: the scheduler now honours every task's effective priority, not only the running one's) and about 550 bytes of flash; the later steps are flat.

9.9 The older timing test

tc04 measures with a 4 μs counter and runs on every board including the UNO; it is coarser but confirms the picture: 24 μs per switch and 24 μs worst interrupt latency on both AVR boards, 1.7 μs and 2 μs on the ESP32 at 240 MHz (10.9 and 12 μs at the 40 MHz crystal clock before the PLL is enabled), with a periodic jitter of 16 μs on the AVR and none the counter can see on the ESP32.

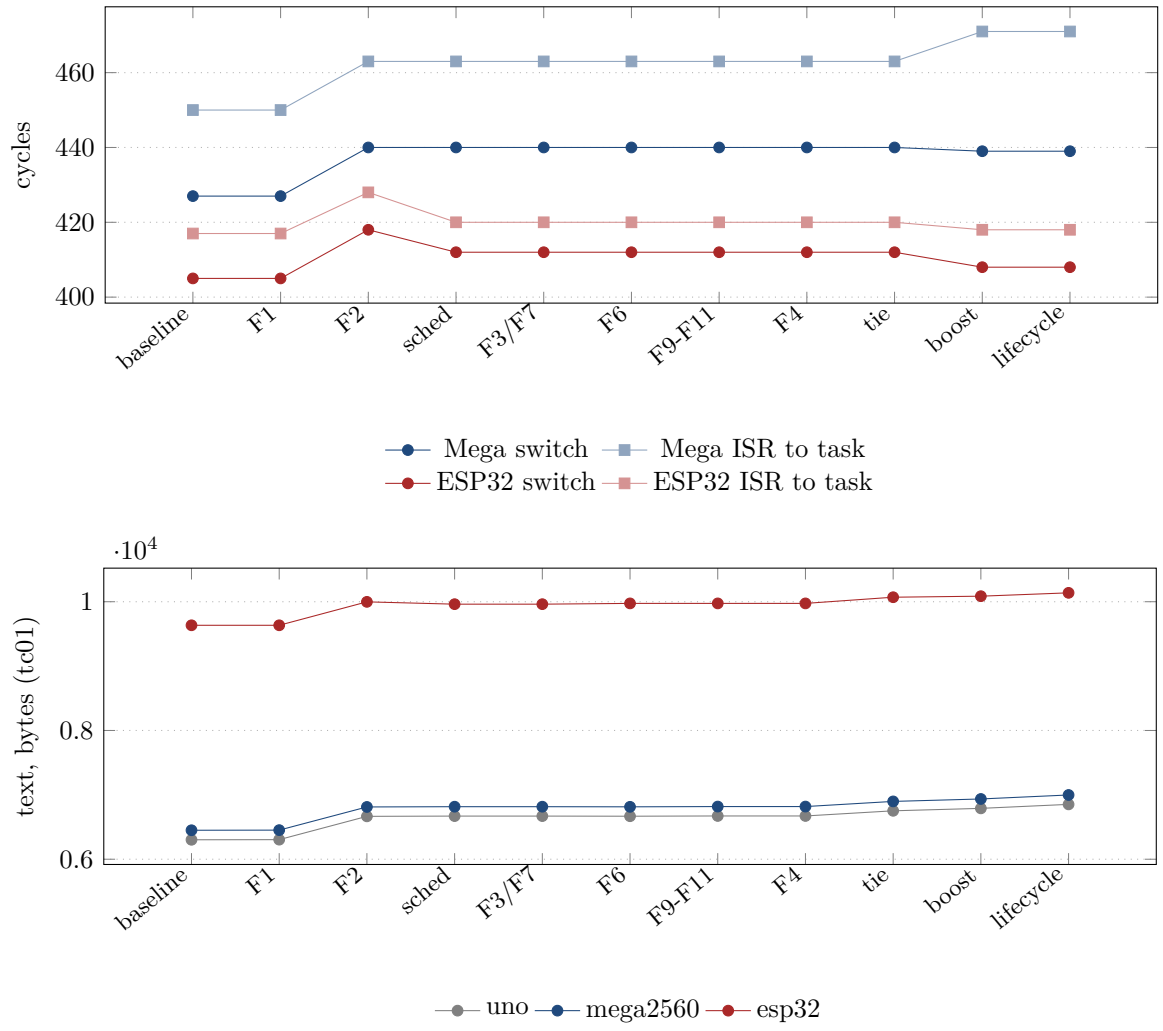


Figure 9.7: The hot path (semaphore switch and ISR to task minimum) and the image size across the measured steps in `doc/measurements/`.

Chapter 10

Verification

Three layers of tests, all run by `make test` from the nornRTOS directory (`tools/test_all.sh` in nornCore; about a minute with an ESP32 connected). Every regression run includes the host tests, simavr for both AVR boards, the ESP32 in QEMU, and the boards that are connected.

10.1 Host unit tests

The host port runs the kernel on `ucontext` with simulated ticks and simulated interrupts, so a test drives every primitive through a scripted sequence of calls and compares the trace of (task, event, value) with the expected one, deterministically. They are built with `-Wconversion`, AddressSanitizer and UndefinedBehaviorSanitizer.

Test	What it pins down
`test_kernel`	three tasks through the scheduler, events, delays, periodic waits, semaphores, mutexes
`test_queue`	queues and mailboxes: hand-over, overflow, timeouts, the ISR variants
`test_notify`	task notifications and software timers, one shot and periodic
`test_pool`	memory pools and the run time statistics
`test_features`	heap, event groups, suspend and resume, the tickless query
`test_timeout`	timeouts at the limit of 16 bit ticks: `RTOS_MAX_TIMEOUT` and beyond, `RTOS_FOREVER - 1`
`test_wrap`, `test_wrap32`	the tick counter running through four wraps (16 and 32 bits) with every timed primitive in use, each interval checked to the tick
`test_prio`	the scheduler against mutex ceilings: an owner that blocked while holding, a waiter handed the mutex, nested ceilings
`test_tie`	the scheduling order at equal effective priority (section 3.6.4) in the task-context pick and in the interrupt tail
`test_lifecycle`	a task whose function returns is terminated and deaf; a reset slot starts a new task clean; a return while holding a mutex panics
`test_robust`	deadlock, stack canary, task watchdog, timer task, statistics, task info, trace, assertion; one child process per scenario
`test_checks`	every mandatory check with assertions compiled out, one child process per violation
`test_api`	the contract test: every public function called as the header documents it, return values and statistics checked
`test_stress`	six tasks pick random operations on every primitive with random timeouts while the idle hook injects random interrupts; the kernel invariants are checked after every operation; 8 seeds × 200 000 ticks

Table 10.1: The host unit tests in `nornCore/test/`.

`make host-cov` builds the host tests with gcov and reports the line coverage per kernel file: 98 % of the kernel, the rest being the reset paths of `rtos_panic.c` that the host cannot take and the stack realignment only the ESP32 needs. `make analyze` runs GCC's static analyser over the kernel, the ports and the HAL with each architecture's compiler and reports nothing.

10.2 Test applications on the targets

Each test case checks its own counters at the end and prints `RESULT tcNN PASS` or `FAIL`, then halts, which ends a simulator run. tc01 to tc06 pass on a real Mega 2560 with the same counters as simavr, on the ESP32 board and in QEMU.

Case	What it runs
tc01	periodic task, events, a semaphore from an ISR, a ceiling mutex, the stack check
tc02	a queue fed from an ISR with deliberate overflow, a mailbox between tasks
tc03	notifications from an ISR and from timers, one shot and periodic software timers
tc04	timing: context switch, interrupt latency, periodic jitter
tc05	panic, reset and recovery: an application panic, a stack overflow, a late task, a starved idle task; the record checked after each reset
tc06	floating point kept in registers across preemption; the lazy FPU context on the ESP32
tc07	ESP32: a second kernel instance on the other core answering requests over the shared memory link
tc08	heap, event groups, suspend and resume, tickless idle together
tc09	ESP32: code and constants running from flash through the cache
tc10	the kernel benchmark in cycles (chapter 9)
tc11	stack need per task
tc12	stack and interrupt entry latency against call depth

Table 10.2: The test applications in `nornCore/applications/`.

10.3 Continuous integration and the simulators

The host tests, simavr and QEMU run on every push. QEMU is not cycle accurate and stalls when the host is busy: its timing figures mean nothing, and a heavy parallel build on the host can fail the latency limits of tc04 and tc10. For stack use QEMU is exact. The rule for every change to the kernel is: tests first, then the change, then `tools/measure.sh` before and after, then the full regression.

Appendix A

Porting in brief

The kernel never touches hardware. Everything it needs comes from `port/<arch>/port.h`, which every kernel file includes through `rtos.h`. A port is three or four files plus a make fragment: the AVR port is the smallest complete example, the host port shows the minimum that makes the kernel run at all, and the Xtensa port shows a CPU with register windows, a lazy FPU and a ROM loader. `porting.md` in nornDocs is the full description; in outline a port provides:

1. **Types and constants**: the tick, event and task mask types, the stack sizes (`RTOS_PORT_MIN_STACK`, `DEFAULT`, `LARGE`, `STACK_WARN`, `STACK_ALIGN`), the run time counter frequency.
2. **Critical sections**: enter returns the previous interrupt state, exit restores it.
3. **The interrupt model**: the `RTOS_ISR` macro and the interrupt tail that switches once at the end of an ISR; the tick interrupt.
4. **The context switch**: initialise a stack so that the first switch to it enters the task trampoline; save the current context and restore another.
5. **Time**: start the tick at `RTOS_CFG_TICK_HZ`; a free running microsecond source for `rtos_getTimeUs`; the run time counter.
6. **Robustness hooks**: halt, reset, the memory that survives a reset for the panic record, the reset cause, the hardware watchdog.
7. **The make fragment**: compiler, flags, the port's sources, how to upload and how to simulate.

Bringing a port up is done with tc01 first (the kernel's primitives on the new CPU) and tc10 last (its cost), and the port is accepted when the host unit tests' invariants hold on the target through tc01 to tc06.

Appendix B

Towards dynamic tasks

nornCore is a static kernel by design, and the team decided in November 2026 to prepare for creating and deleting tasks at run time by formalising the life cycle first (section 3.7) rather than by adding a scheduler feature. The plan is `doc/dynamic_tasks_plan.md` in nornCore; this is its outline so that a reader of this guide knows what to expect.

Done. The scheduling order with a held mutex is a total order on a documented key; the running task's own boost costs nothing on the hot path; a task whose function returns is `RTOS_TASK_TERMINATED`, deaf and checked against holding a mutex; one function produces the initial state of a task slot, used by `rtos_init` today and by the reap tomorrow. The test `test_lifecycle` exercises all of it, including a slot reset and re-created under a critical section.

Remaining, in order. A public `rtos_exit()` so that a task can end without returning through its whole call chain; a count of held mutexes in the TCB, which closes the one hole in the exit check (a mutex whose ceiling equals the owner's priority raises nothing); a reap callable on a terminated task only, from another context, with a port hook that frees the stack where a port allocated one; a run time create that wraps `rtos_createTask` in a critical section and a schedule; and a configuration option, off by default, under which all of it is compiled, with the proof that a static system's kernel is byte for byte the one it is today.

Never. Killing a READY, BLOCKED or RUNNING task from outside: the task may sit in the timed list, in waiter masks, own a mutex or be executing on the very stack that would be freed, and that operation is behind most use-after-free bugs in kernels that offer it. Task ids stay equal to priorities; a reused slot gives the new task the priority of the slot.

Bibliography

[1] P. Vranken. *RTuinOS 1.0 User Guide*. A real time operating system for Arduino, 2013. `https://github.com/PeterVranken/RTuinOS`.

[2] C. L. Liu and J. W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.

[3] L. Sha, R. Rajkumar and J. P. Lehoczky. Priority inheritance protocols: an approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.

[4] T. P. Baker. Stack-based scheduling of realtime processes. *Real-Time Systems*, 3(1):67–99, 1991.

[5] A. Burns and A. Wellings. *Real-Time Systems and Programming Languages*, 4th edition. Addison-Wesley, 2009.

[6] Y. Yamamoto. *microps*: an implementation of a small TCP/IP protocol stack for learning. `https://github.com/pandax381/microps`.

[7] W. Eddy (ed.). Transmission Control Protocol (TCP). RFC 9293, IETF, 2022.

[8] Microchip Technology. *ATmega640/1280/1281/2560/2561 Datasheet* and *ATmega328P Datasheet*.

[9] Espressif Systems. *ESP32 Technical Reference Manual*, and the *ESP-IDF Programming Guide* for the radio libraries' operating system adapter.

Index

rtos_broadcastEvents, 31
rtos_createTask, 30
rtos_delay, 30
rtos_evgroupInit, rtos_evgroupSet, rtos_evgroupSetFromIsr, rtos_evgroupClear, rtos_evgroupGet, 35
rtos_evgroupWait, 35
rtos_getCurrentTask, 36
rtos_getLastPanic, rtos_getResetCause, 37
rtos_getOverrunCount, 36
rtos_getStackFree, 36
rtos_getTaskInfo, 37
rtos_getTaskRunTime, rtos_getCpuLoad, 37
rtos_getTick, 30
rtos_getTimeUs, 37
rtos_heapInit, rtos_malloc, rtos_free, rtos_heapStats, 36
rtos_idleHook, rtos_panicHook, rtos_stackWarnHook, rtos_assertFailed, rtos_traceHook, 37
rtos_init, 29
rtos_mboxInit, rtos_mboxPost, rtos_mboxPostFromIsr, rtos_mboxFetch, rtos_mboxIsFull, 33
rtos_mutexInit, 32
rtos_mutexLock, 32
rtos_mutexUnlock, 32
rtos_notify, rtos_notifyFromIsr, 33
rtos_notifyTake, 34
rtos_notifyWait, 34
rtos_panic, 37
rtos_poolAlloc, rtos_poolAllocFromIsr, 35
rtos_poolFree, rtos_poolFreeFromIsr, rtos_poolFreeCount, 35
rtos_poolInit, 35
rtos_postEvents, rtos_postEventsFromIsr, 31
rtos_queueCount, 33
rtos_queueInit, 32
rtos_queueReceive, rtos_queueReceiveFromIsr, 33
rtos_queueSend, rtos_queueSendFromIsr, 33
rtos_semGive, rtos_semGiveFromIsr, 32
rtos_semInit, rtos_semInitMax, 31
rtos_semTake, 31
rtos_start, 30
rtos_suspend, rtos_resume, rtos_resumeFromIsr, 36
rtos_timerInit, 34
rtos_timerStart, 34
rtos_timerStop, rtos_timerIsActive, rtos_timerRemaining, 34
rtos_waitEvents, 31
rtos_waitPeriodic, 30
rtos_wdtInit, rtos_wdtMonitor, rtos_wdtCheckIn, 38